

北京理工大学

本科生毕业设计(论文)

Alien 内核的 x86_64 移植与 vDSO 引入

Alien Kernel: x86_64 Porting & vDSO Integration

学 院:	计算机学院
专 业:	计算机科学与技术
班 级:	07022202
学生姓名:	胡嘉晨
学 号:	1120220611
指导教师:	陆慧梅

2026 年 5 月 26 日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：胡嘉晨 日期：2026年5月26日

关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：①学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；③学校可允许本毕业设计（论文）被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；⑤学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：胡嘉晨 日期：2026年5月26日

指导老师签名：陆慧楠 日期：2026年5月26日

Alien 内核的 x86_64 移植与 vDSO 引入

摘 要

传统内核因为各个模块高度绑定，一处故障很容易扩散到整个系统，导致可靠性降低和维护难度增加。Alien操作系统通过将内核功能拆分为相互隔离的“故障域”，并使用Rust语言的安全机制，在操作系统层面提供更高的容错并缩小漏洞影响范围。x86_64架构在服务器和桌面领域使用广泛、生态成熟，且用户态中断机制已在真实硬件上落地。移植能更好地验证隔离设计的可用性，同时引入新的软件和硬件特性。x86_64在引导、中断控制、Linux ABI与设备发现等机制上与RISC-V存在大量差异，移植到x86_64需要修改整体框架以支持多架构扩展，并添加大量x86_64细节。

为此，本文先分析了Alien原始设计中的域模型，随后设计并实现了x86_64的启动流程、trap/syscall入口、内存管理及设备探测与驱动加载框架。在此过程中，本文提炼出了Trap分发机制等公共抽象，让架构差异仅存在于架构实现内部，使得主逻辑可以跨平台。在此基础上，本文进一步设计并集成了vDSO机制：在保证系统调用路径仍然保留的前提下，通过将一块只读的代码与数据区域映射至用户地址空间，使用户程序获取当前时间可以绕过系统调用。

本文完成了AsyncAlien在x86_64上的包含启动、多核同步、设备发现、域加载及进入用户态的全流程，验证了功能正确性。性能测试结果显示，引入vDSO后，clock_gettime等时间查询操作的开销降低在90%以上。本工作不仅完成了Alien架构移植与功能扩展，也为构建多架构的内核提供了设计参考与实现经验。

关键词：操作系统；多架构支持；内存安全；vDSO；系统调用

Alien Kernel: x86_64 Porting & vDSO Integration

Abstract

Traditional kernels suffer from high module coupling, where a single failure can easily propagate throughout the system, reducing reliability and increasing maintenance difficulty. The Alien operating system enhances fault tolerance at the OS level by partitioning kernel functions into isolated "fault domains" and leveraging Rust's safety mechanisms to limit the scope of vulnerabilities. The x86_64 architecture is widely used in servers and desktops with a mature ecosystem, and its user-space interrupt mechanism has been implemented on real hardware. Porting allows for better validation of isolation design feasibility while introducing new software and hardware features. However, x86_64 differs significantly from RISC-V in mechanisms like booting, interrupt control, Linux ABI, and device discovery. Porting to x86_64 requires modifying the overall framework to support multi-architecture extensions and incorporating extensive x86_64-specific details.

To this end, this paper first analyzes the domain model in the original design of Alien, then designs and implements the x86_64 boot process, trap/syscall entry, memory management, and device detection and driver loading framework. During this process, it extracts common abstractions such as the Trap dispatch mechanism, ensuring architectural differences exist only within the implementation of the architecture, allowing the main logic to be cross-platform. Building on this, the paper further designs and integrates the vDSO mechanism: while maintaining the system call path, it maps a read-only code and data region to the user address space, enabling user programs to bypass system calls when retrieving the current time.

This paper completes the full process of AsyncAlien on x86_64, including booting, multi-core synchronization, device discovery, domain loading, and entry into user mode, verifying the correctness of the functions. Performance test results show that with the introduction of vDSO, the overhead of time-related operations such as `clock_gettime` is

reduced by above 90%. This work not only achieves the porting and functional expansion of the Alien architecture but also provides design references and implementation experience for building a multi-architecture kernel.

Key Words: Operating System; Multi-architecture Support; Memory Safety; vDSO; system call

目 录

摘 要	I
Abstract	II
第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 研究目标与问题陈述	2
1.3 本文的组织结构	2
第 2 章 背景与相关工作	4
2.1 原始 Alien 系统综述	4
2.2 微内核与隔离设计对比	6
2.3 Rust 语言的内存安全	7
2.4 vDSO 与高频路径优化	8
第 3 章 系统架构与设计	11
3.1 引入 x86_64 并屏蔽多架构差异	11
3.1.1 引导与早期初始化	12
3.1.2 内存管理	13
3.1.3 中断控制	13
3.1.4 设备探测与域加载	14
3.2 vDSO 设计	15
3.2.1 vdso_crate_template 设计	15
3.2.2 vDSO 与 syscall 设计	17
第 4 章 x86_64 移植与实现细节	19
4.1 引导与早期初始化	19
4.2 x86_64 与 RISC-V 的页表结构与寄存器差异	21
4.3 trap/syscall 初始化	22
4.4 ACPI 与 PCI 的探测方式与域加载	25
第 5 章 vDSO 实现与用户态集成	29
5.1 vdso_crate_template 构建与产物	29
5.2 内核侧 MemIf 实现与 vDSO 装载	32

5.3 内核侧刷新策略	33
5.4 用户运行时 vDSO 加载	35
第 6 章 验证与评估	37
6.1 测试设计	37
6.2 功能验证与运行证据	40
6.3 性能度量与分析	41
结 论	44
参考文献	45
致 谢	47

第1章 绪论

1.1 研究背景与意义

随着计算机系统功能日益复杂，Linux内核代码规模也在不断增长。这种增长使得内核的开发和维护工作变得越来越繁重。传统宏内核架构将操作系统的大部分功能集中在单一的执行环境中，任何一个模块出问题，都可能影响整个系统的安全性和可靠性。从多个真实漏洞来看，传统内核的复杂性不只是代码量大、逻辑耦合紧，更导致漏洞修复代价高、验证困难、攻击面宽且难以收敛。这些漏洞案例提醒我们，仅仅依靠打补丁的方式很难彻底解决内核复杂带来的根本问题。

为应对上述风险，Linux内核社区提出了严格的内核内存权限控制、KASLR^[1]、seccomp^[2]、模块签名等防御手段。这些机制有效提高了攻击门槛，但其本质仍是对复杂系统的被动补救，难以从根本上降低内核复杂性带来的风险。换句话说，这些方法虽然有用，但都是在已有架构上打补丁，并没有改变宏内核本身的结构。隔离式内核架构由此成为一种可行的解决方案：通过把系统功能划分为相对独立的故障隔离域，各域通过明确接口交互，避免直接共享内核内存，从而将故障影响局限在域内并减少受影响的内核路径。这种思路为降低内核复杂性提供了新的方向。

在这种架构下，系统具备如下优势：

1. 单个域的漏洞或故障不会导致整个系统崩溃，即使某个域出现问题，其他域仍然可以继续正常运行。
2. 每个域只暴露必要的接口，减少高风险路径，这样可以大幅降低攻击者能够触及的内核代码范围。
3. 功能模块可以独立升级或替换，降低系统整体维护成本，不同模块的开发者之间也不会相互干扰。
3. 安全属性更易于验证与审计，因为每个域之间的交互都是通过固定接口进行的，容易追踪和检查。

Alien是基于域隔离理念的内核实现，已在RISC-V平台验证了隔离域、共享堆通信与域加载等机制。x86_64平台在启动、trap处理、内存和设备管理等方面与RISC-V存在显著差异，需要设计平台适配层。系统调用、时间查询等高频路径对性能尤为敏感，传统Linux通过vDSO在用户态提供快速路径以降低开销。任务调度框架作为操作

系统的核心组件之一，对于任务整体性能有着至关重要的作用。为了引入基于vDSO的调度框架，需要先引入vDSO以及快速syscall路径作为前置工作。

1.2 研究目标与问题陈述

本文的研究目标是：在不动现有RISC-V路径的前提下，打通x86_64的运行链路，使系统能够完成启动、早期初始化、设备发现、域加载、多核启动并最终进入用户态；同时为vDSO、协作调度等高频机制预留好扩展接口，让这个最小可运行版本后续有优化余地。这个目标要求既要保证基本功能正确，又要为将来性能提升留下空间，不能为了快速实现而牺牲扩展性。

围绕这个目标，本文需要回答以下2个问题：

1. 如何在启动流程、内存布局、中断处理、设备探测机制等方面建立统一的抽象，使x86_64与RISC-V之间实现尽可能多的代码复用，同时保持语义和功能 correctness？这个问题的关键在于找到两种平台的共同点，把差异部分隔离到独立的模块中。
2. 如何将vDSO引入Alien，同时不影响现有的域隔离、系统调用等机制，实现一个功能正确的时间获取调用？这个问题的难点在于vDSO需要在内核和用户态之间共享只读数据，又不能破坏域隔离的安全边界。

本文的研究范围就限定在上面这些问题在x86_64平台上的实现和验证。最后会从功能正确性和性能开销这2个维度，说明迁移后的系统不仅能启动，而且具备一定的实际可用性。本文会检查系统是否按预期工作，同时也会评估它在运行时的速度开销是否在可接受范围内。

1.3 本文的组织结构

本文后续章节按背景、实现、验证的顺序展开。

第二章介绍Alien的原始设计以及相关研究进展，包含域隔离、共享堆通信、vDSO和协作调度等核心概念，为后面的设计提供理论基础。

第三章给出系统整体架构与设计要点，说明平台抽象、虚拟地址空间和高频路径的统一设计思路，以及如何在设计中为vDSO和协作调度预留扩展接口。

第四章重点讲x86_64的移植和实现细节，依次说明引导与早期初始化、物理与虚拟内存管理、多核启动同步、trap与syscall的适配、任务切换、设备发现以及驱动域加载的工程实现和关键取舍。

第五章描述vDSO的构建、vVAR布局与无锁一致性策略、用户态映射与auxv注入，以及快路径和回退机制的协同实现。

第六章给出实验验证与评估方法，包括功能正确性测试、隔离性与恢复性验证，以及针对syscall和vDSO的性能基准测试与分析。

第2章 背景与相关工作

2.1 原始Alien系统综述

Alien^[3]的设计思路不是简单地对内核进行模块划分，而是按照职责和功能，把内核能力拆成若干个独立的故障隔离域。这些域之间通过显式定义的接口相互调用，共同完成系统功能。在传统的宏内核架构中，各个模块运行在同一个高权限的执行空间里，任何模块出现内存错误或逻辑缺陷都可能直接破坏整个内核。与此不同，Alien在框架设计上就直接把访问边界、调用边界和故障边界划分清楚。这样一来，当系统某个局部发生问题时，故障被限制在单个域内部，不会轻易扩散到其他部分，大多数情况下只需要修复或替换出问题的域即可。本文所做的迁移工作，必须保持这一设计不受到破坏，并优先复用那些在原型中已经得到验证的域运行时机制。

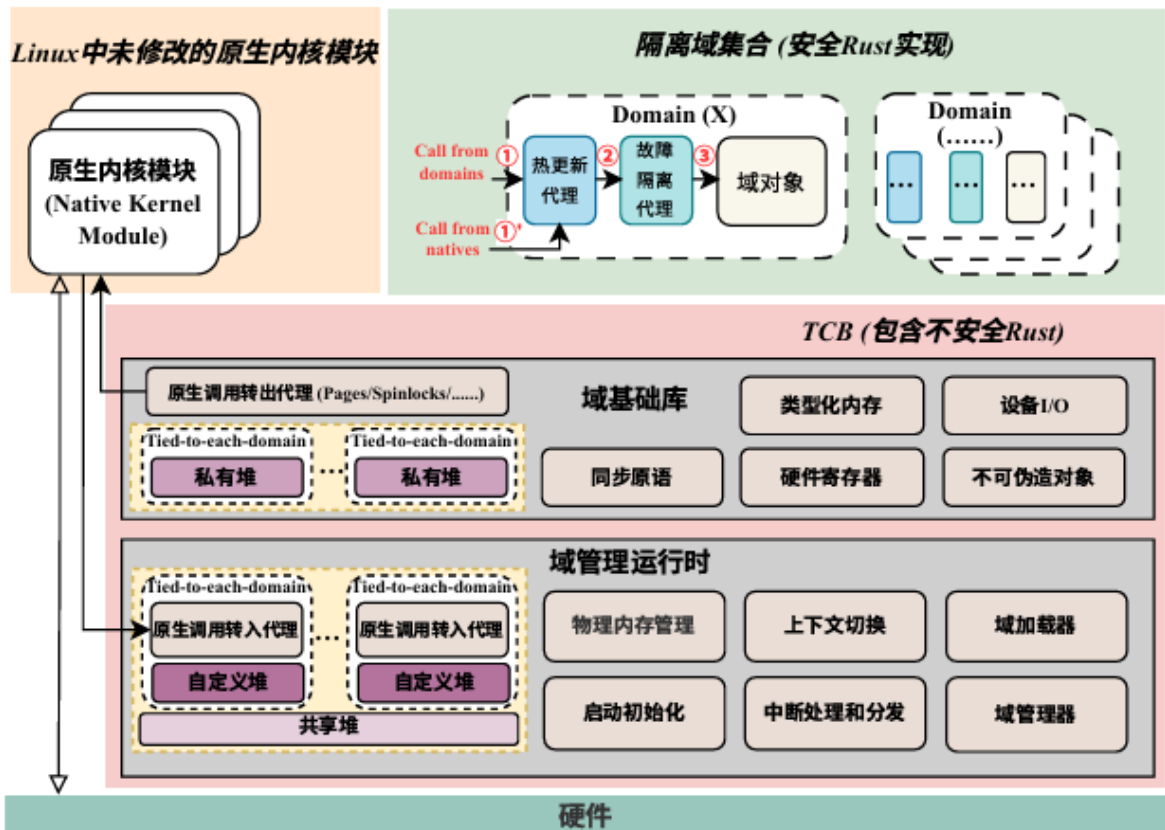


图 2-1 Alien 整体架构图

从运行的角度看，**Alien**的内核功能并不是在启动时一次性全部创建好，而是以一组可动态管理的隔离域形式存在。这些域相当于独立的内核模块，能够在系统运行过程中被加载、卸载或热更新。每个域通过域接口对外提供服务，域接口定义了该域支持哪些功能以及需要传入什么类型的参数。域内部的功能实现则依赖于安全抽象，这些安全抽象来自可信计算基，封装了对物理内存、设备寄存器等底层资源的不安全操作，对外暴露安全的调用方式。域管理运行时负责域的装载、回收和替换等治理动作，它在系统运行过程中持续工作，使得整个内核形成“加载—通信—治理”这样一个闭环。该闭环的核心目标就是实现隔离、故障隔离和持续演进的能力：借助**Rust**语言的类型和内存安全特性以及清晰的接口边界，每个域可以独立升级和替换，同时又不会让跨域协作退化成难以理解和验证的隐式耦合。

在域间通信方面，**Alien**采用了控制面和数据面分离的方式。控制路径负责传递调用意图和少量参数，使用受限的接口和轻量级函数调用，保证调用过程简单且开销低。数据路径则用于交换较大的数据块，它通过一个所有域都可以访问的共享堆来传递对象。域A要向域B传递数据时，需要在共享堆上分配一块内存，把数据放进去，然后将这块内存的所有权通过域接口传递给域B。移交所有权后，域A就不再持有对该数据的访问权限，这由**Rust**的所有权系统在编译时自动检查。共享堆上使用的数据类型有严格限制，不能包含指向域私有堆的指针，也不能存储私有堆上的对象。这种设计兼顾了性能和边界可验证性：一方面，高频数据交换时不需要反复拷贝数据，通信效率较高；另一方面，通过类型约束和生命周期检查，从语言层面避免了越界访问、释放后使用等常见的内存安全问题。**Alien**不依赖开发者之间的人工约定来保证通信正确性，而是尽可能把约束落实到接口和类型边界上，让编译器去检查和执行。

不过，原始**Alien**的实现主要面向RISC-V平台，因此它的启动过程、异常入口、设备发现以及中断处理模型都带有明显的平台特征。例如，启动时依赖**SBI**接口获取硬件信息并完成模式切换，异常和中断通过统一的向量入口进行分发，设备信息则通过设备树描述。这些设计在原型阶段有利于快速验证域机制的可行性，但也使得部分实现与具体的平台细节耦合得比较深。本文对**Alien**进行x86_64迁移，并不是要推翻原有设计重新开始，而是在保留域机制主干部分的前提下，把平台相关的差异从上层运行时中分离出去。具体来说，就是把域管理运行时和域基础库划分为系统无关部分和系统相关部分，系统无关部分可以直接复用，系统相关部分则针对x86_64进行重新实现或适配，例如启动引导方式、中断描述符表、**ACPI**设备枚举以及页表结构等。

通过这种方式，逐步建立一套可复用的跨架构实现骨架，使Alien的隔离架构能够在更多平台上运行。

2.2 微内核与隔离设计对比

微内核体系^[16]（如seL4^[15]）通过最小化内核功能来压缩可信计算基，并依赖消息传递实现服务协作，隔离边界天然更强。从设计粒度看，微内核追求最小内核，通常仅保留地址空间、线程和进程间通信等最基础的服务，其他功能以用户态服务进程的形式运行。这种设计虽能提供强隔离，但会引入上下文切换与消息传递的额外开销。

微内核的设计遵循两项核心理论。其一是“最小化原则”：只有那些一旦移出内核便会系统无法正常工作功能，才应保留在内核内部。据此，内核仅需抽象三类基本资源——物理内存（地址空间）、CPU（线程）以及中断与异常。至于内存分配策略、调度策略、文件系统实现等，均属于策略范畴，应交由用户态服务进程负责。内核只提供最基础的机制（如创建地址空间、启动线程、发送消息），从而将代码量控制在很小规模，有效降低缺陷密度与攻击面。

其二是“机制与策略分离原则”。内核仅负责执行机制，即“如何完成某项操作”，而不参与策略决策，即“应该执行何种操作”。例如，内核提供消息传递的通道（机制），而消息的访问权限、目标对象、发送频率等策略约束由用户态服务自行决定。将策略移出内核后，策略更新无需重启内核；即使策略模块崩溃，也不会导致整个系统失效。同时，内核无需理解复杂的安全规则，攻击面因此进一步缩减。

上述两项理论共同作用，最小化原则指明了哪些功能不得进入内核，机制与策略分离原则则要求即使保留在内核的功能也不能内嵌策略。遵循这套设计路径，微内核的进程间通信（IPC）可被高度优化——消息长度通常限制在寄存器能直接容纳的范围内，大多数通信无需访问内存，仅需十余条机器指令即可完成一次跨地址空间的过程调用。

对本研究而言，微内核提供了隔离设计的参照系与终极目标。然而在工程实践中，将全部系统服务一次性迁移至用户态往往面临较高的重构成本。与此相对，Alien路线更具工程折中色彩：不追求一次性重写所有系统服务，而是在现有内核形态上逐步引入隔离域和显式接口，采用“渐进式隔离重构”策略。Alien保留了大部分内核功能在内核态执行，但通过域机制强化模块间的边界与可验证性，在隔离效果与性能

开销之间取得平衡。两者的共同目标是降低故障扩散与攻击面，区别在于实施路径与成本结构。**Alien**路线强调迁移可行性与渐进演化能力，更契合现有代码库基础与工程实践约束。这一权衡决定了后续x86_64迁移的实施策略：优先复用现有运行时框架，在其上引入平台抽象和隔离边界，而非彻底推翻重来。

2.3 Rust 语言的内存安全

Rust是一门系统级编程语言，它的设计目标是在不牺牲性能的前提下保证内存安全。与**Java**、**Go**等依赖垃圾回收（GC）来管理内存的语言不同，**Rust**不需要运行时垃圾回收，因而没有暂停开销，能够胜任操作系统内核、嵌入式设备等对实时性要求严格的场景。**Rust**的内存安全完全建立在编译时的静态检查之上，其核心机制包括所有权、借用和生命周期。这三者协同工作，在代码编译阶段就完成所有内存访问的正确性验证，既避免了手工管理内存带来的各种错误，又保持了与**C/C++**相当的执行效率。

所有权是**Rust**内存安全体系的基石^[18]。每个值在任意时刻有且只有一个变量作为它的所有者；当所有者离开作用域时，该值所占用的内存会被自动释放。所有权可以通过移动语义转移：将一个值赋值给另一个变量时，原变量即失去对该值的访问权，编译器会禁止后续使用。这种机制彻底杜绝了双重释放——因为释放责任被唯一地绑定到当前所有者；同时也防止了释放后使用——因为原所有者已经无法访问该值。传统**C**语言要求程序员显式调用释放函数并自行维护指针有效性，这种手工管理极易出错；而**Rust**的所有权规则将这些责任固化在编译检查中。

借用机制允许在不转移所有权的前提下临时访问一个值^[19]。借用分为两类：不可变借用和可变借用。编译器强制规定：在任一时刻，要么只能存在一个可变借用，要么可以存在任意数量的不可变借用，但两者不能同时出现。这一规则在编译期就排除了数据竞争的可能——多个读取操作并行安全，但写入操作期间不允许任何其他访问。不可变借用禁止通过该引用修改原值，从而防止了迭代器失效等问题；可变借用则赋予独占的修改权限，在此期间原值被冻结。借用检查器会静态分析代码的所有执行路径，若违反上述规则则直接拒绝编译，从根源上消除了数据竞争。

生命周期机制用于确保所有引用都是有效的，防止悬垂引用^[19]。每一个引用都附带一个生命周期参数，表示该引用所指向的数据从创建到销毁的时间段。借用检查器在编译时会比较不同引用的生命周期范围：如果一个引用被用于访问数据，编译器

必须能够证明被引用的数据在该引用存续期间始终有效。当生命周期关系无法被自动推断时，开发者可以使用生命周期标记进行显式标注，以帮助编译器建立约束。生命周期标注不改变运行时的行为，只用于编译期的静态验证。生命周期与借用的结合使得Rust能够安全地返回指向栈上数据的引用、管理容器中的引用成员等，而这在传统的C/C++中往往是未定义行为的高发区。

所有权、借用与生命周期三者并非孤立的规则，而是一个统一的静态推理系统。所有权确立了资源的唯一管理者；借用规则控制了并发访问的模式；生命周期则划定了每个引用的有效时间窗口。RustBelt^[17]利用分离逻辑在Coq定理证明器中完成了对该系统安全性的机械化形式化验证，证明了Rust类型系统在所有安全代码（即不使用unsafe关键字的部分）中确实禁止了内存错误和数据竞争。这一证明覆盖了所有权的移动语义、借用检查器的限制条件以及生命周期子类型关系等核心语言特性，从而为Rust的内存安全提供了坚实的数学基础。

当然，Rust并不表示底层代码就可以完全摆脱不安全操作。域加载、地址空间映射、硬件直接交互等路径仍然需要少量unsafe代码，这些代码触及底层内存和硬件语义，已经超出了Rust编译器安全覆盖的范围。相关实践的意义在于表明：Rust能够将绝大多数可能引入错误的代码压缩到少数几个可集中审计的边界内，从而使隔离域系统的工程质量更加可控，而不是宣称“语言本身会自动解决所有问题”。这样的分层设计让后续的代码审计、测试与形式化验证都能够更加聚焦有效。

2.4 vDSO 与高频路径优化

vDSO（virtual dynamic shared object）^[14]是Linux内核为降低高频系统调用开销而引入的一项经典优化机制。传统系统调用的性能瓶颈主要体现在用户态与内核态之间的上下文切换开销——每次陷入内核都需要保存和恢复大量寄存器状态，这一固定成本在调用频率极高时将累积为不可忽视的性能损失。vDSO的设计目标正是在保持语义正确的前提下，让某些无需修改内核状态的系统调用完全在用户态执行，从而消除陷入内核的开销。

vDSO的起源可以追溯到2005年前后Linux 2.6内核引入的vsyscall机制，后者是Linux最早的加速系统调用的尝试，出现在2.6内核中，通过将少量内核代码以静态页面的形式映射到用户地址空间来实现快速调用。然而vsyscall存在若干固有缺陷，其采用固定静态地址映射，缺乏灵活性，也无法为不同进程提供差异化的映射策略。作

为改进方案，vDSO在2007年由Andi Kleen首次在x86_64架构上实现^[20]，内核将一小段ELF共享目标文件自动映射到每个用户进程的地址空间，该文件包含了`gettimeofday`和`clock_gettime`等高频系统调用的用户态实现。相较于`syscall`的静态内存映射方式，vDSO借助ELF共享目标文件天然的PIC（位置无关代码）特性，以进程为单位动态映射到不同地址空间中，同时为每个进程随机化映射地址以增强安全性。

vDSO在性能上带来的提升是显著的。以经典的`gettimeofday`调用为例，在s390平台上进行的一亿次调用基准测试中，纯系统调用路径耗时8.6秒，而vDSO路径仅需1.0至1.3秒，性能提升约6至8倍^[22]。在arm64平台上，arm64vDSO的引入使`clock_gettime_monotonic`的耗时降至原先的23%，`gettimeofday`降至26%，其他时钟相关调用也有3倍以上的改善。在x86_64平台上，`CLOCK_MONOTONIC`路径的vDSO调用平均延迟约25纳秒且无需陷入内核，而强制走系统调用路径则需约320纳秒，差异超过一个数量级。

随着vDSO在多架构上的广泛部署，其实现分散在各架构目录中也带来了维护负担。2019年，Vincenzo Frascino在Linux 5.2开发周期中提交了通用vDSO实现^[21]，将大量架构相关代码重构为统一的通用层，使vDSO可以跨越x86、arm64、s390、MIPS等多种架构共享同一套核心逻辑。这一工作的意义不仅在于减少了代码重复，更重要的是确立了vDSO的跨架构设计模式：内核将只读的vVAR数据页映射到用户空间，vDSO代码通过读取这些数据页中的预更新值（如时钟调整参数）来完成快速计算，从而在不进入内核的情况下返回正确结果。

与此同时，vDSO的安全性始终是学术界和工业界关注的焦点。从安全模型来看，vDSO代码完全运行在用户空间，并不跨越任何安全边界；当需要触及内核特权操作时，vDSO必须通过系统调用回退到内核执行。然而历史上vDSO仍暴露出若干安全问题。早期vDSO映射地址固定，曾被用于绕过ASLR（地址空间布局随机化）保护，攻击者可通过猜测vDSO的固定地址来构造ROP链。此外，vDSO区域还可能被用作潜在的KASLR侧信道信息泄露源，攻击者可通过推测侧信道从vDSO映射地址推导内核基址。vDSO劫持技术甚至被用于容器逃逸，例如DirtyCOW漏洞通过篡改vDSO代码突破容器隔离获取宿主机root权限^[24]。针对这些安全威胁，内核社区持续加固vDSO的安全防护：通过地址随机化消除固定地址攻击面；在Linux 6.15中引入`mseal`机制，将`vdso`、`vvar`、`sigpage`等系统映射锁定为只读或仅可执行，确保它们在进程生命周期内不被篡改或解除映射^[23]。

从vDSO的技术演进中，可以提炼出几条具有普遍参考价值的设计原则。其一，“快慢路径协同”——高频、无状态、低风险的操作优先通过用户态快速路径完成以降低延迟，当快速路径条件不满足或一致性检查失败时，则无缝回退到系统调用慢路径以保证语义正确性。其二，快速路径与主链路不可割裂——若vDSO快速路径与调度器维护的系统时间源不同步，可能导致应用层观察到时间倒流或跳跃，因此快速路径的正确性不仅取决于自身算法，还要求与整个系统的时间管理、调度和一致性框架紧密协同。其三，安全加固应嵌入机制设计而非事后弥补——地址随机化、内存密封等措施已经证明，将安全约束固化到vDSO的基础机制中远比打补丁更可靠。

对Alien而言，vDSO不能被视为独立优化补丁，而应与域隔离、系统调用回退和调度语义协同设计。若快速路径与主链路割裂，可能带来语义偏差甚至边界绕过风险。例如，若vDSO中的时间查询与调度器维护的系统时间不一致，可能导致应用层观察到时间倒流；若vDSO快速路径绕过了域隔离边界，则也就破坏了隔离承诺。因此，vDSO的正确性不仅要求快速路径算法本身正确，还要求与整个系统的隔离、调度和一致性框架协同。

本研究借鉴vDSO思想的重点并非“照搬实现”，而是建立“快慢路径协同”的设计原则。这一原则意味着：高频、低风险、状态不变的服务优先走用户态快速路径以降低延迟；当快速路径条件不满足（如需要修改状态或缺乏必要信息）或一致性检查失败时，必须无缝回退到系统调用慢路径并由内核完成。这样的设计既保证了性能收益，也保证了语义正确性和域隔离不被绕过。该原则为后续x86_64实现章节提供了性能与正确性并重的设计基线。

第3章 系统架构与设计

本文在这一章给出总体架构和关键设计要点，目标是在尽可能屏蔽架构差异的情况下引入x86_64平台相关实现，完成x86_64运行链路，然后在此基础上实现vDSO的引入和测试。这一部分的设计参考了一个面向嵌入式场景的跨平台操作系统Arceos^[7]。

3.1 引入 x86_64 并屏蔽多架构差异

设计目标是把架构相关的差异限制在可控的几处，从而使上层业务逻辑在统一的抽象之上工作。为此把移植工作拆分为四部分：引导与早期初始化、内存管理、中断控制、设备探测与域绑定。每一部分收敛平台差异，向上层提供统一接口，内部按架构实现。

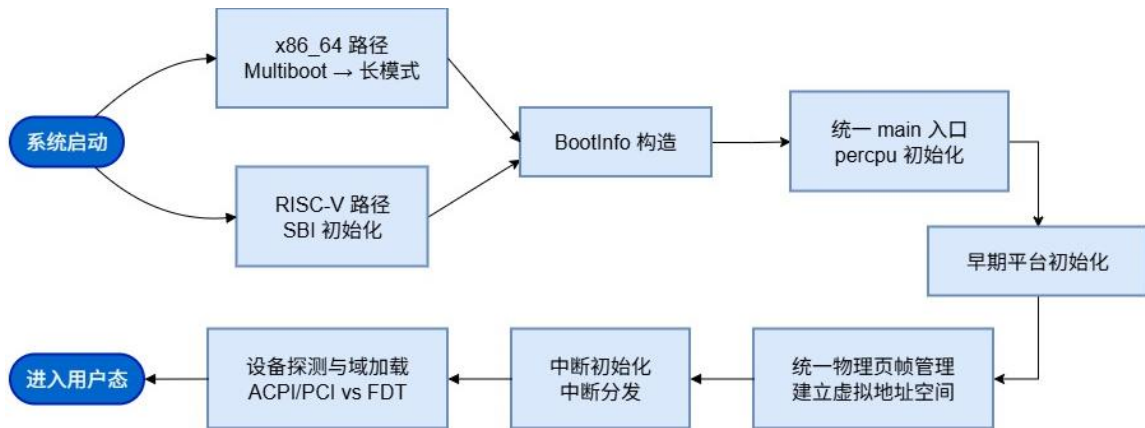


图 3-1 架构差异屏蔽与统一抽象设计图

如图3-1所示，为在系统中引入x86_64支持并尽可能减少对现有代码的影响，本文采用“统一接口、架构内部分化实现”的设计思路。不同架构首先通过各自的启动机制获取平台信息，其中RISC-V侧通过SBI和设备树（FDT）获取启动参数^[13]，x86_64侧通过Multiboot和ACPI获取系统信息^[9]。随后，这些架构相关的数据被转换为统一的BootInfo结构，并作为参数传递给统一的Rust入口函数，从而完成启动阶段的收敛。

在进入统一入口后，系统按照早期初始化、内存管理、中断控制以及设备探测与域加载四个阶段逐步完成运行环境构建。其中，早期初始化负责建立处理器运行所需

的基础环境，包括percpu数据、处理器标识以及平台相关的早期硬件初始化；内存管理通过统一的页帧分配器和虚拟地址空间抽象屏蔽不同架构页表结构和寄存器语义的差异；中断控制模块负责统一异常、中断和系统调用处理流程，并在内部实现不同架构对应的上下文切换机制；设备探测与域加载则负责隐藏FDT、ACPI以及PCI等平台差异，为上层提供统一的设备信息和驱动加载接口。

通过将架构相关逻辑限制在上述四个边界模块内部，上层业务逻辑无需直接访问架构相关寄存器、页表结构或设备发现机制，而是通过统一抽象完成资源管理与服务调用，从而实现跨架构的代码复用与系统可维护性。

3.1.1 引导与早期初始化

引导层的策略是让不同平台通过不同的启动汇编进入统一的Rust编写的主函数，并传递引导信息指针。具体做法是约定一个统一的boot_info参数，传递物理内存布局、initrd基址、可用内存信息等数据，留给后续部分使用。这个信息以指针的形式存在，在不同平台上可能有不同的实际含义。在x86_64上，Multiboot^[12]引导完成后处于32位保护模式，需要设置页表，设置CR0、CR4、EFER控制寄存器进入长模式，然后再进入main；而RISC-V侧只需要在SBI初始化完成后压入boot信息指针后进入main即可。随后双架构初始化percpu数据寄存器（x86_64上是gs，RISC-V上是gp）并进入早期初始化。x86_64侧在早期初始化中设置APIC打开本地中断、启用浮点/SSE功能，建立早期时间模块。而RISC-V得益于其简洁的设计，不需要进行相关工作。

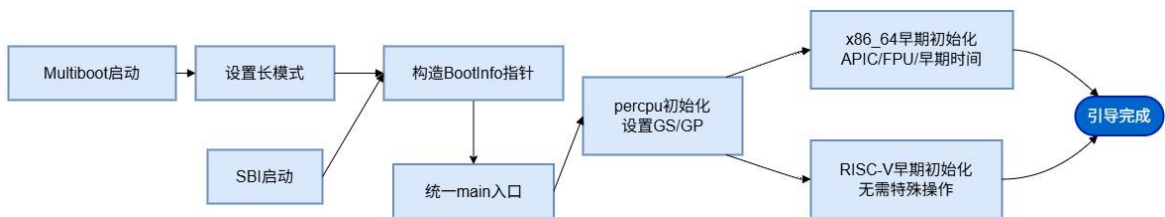


图 3-2 引导与早期初始化跨架构设计

图3-2描绘了引导与早期初始化的跨架构设计。引导阶段将x86_64和RISC-V的不同启动路径分别封装，通过统一的boot_info指针和main入口收敛差异。进入main后，双架构均进行percpu初始化，随后执行各自的平台早期初始化，其中x86_64需要配置

本地中断和浮点环境，而RISC-V无需此类操作。该设计将架构相关步骤限制在边界内，上层逻辑可复用。

3.1.2 内存管理

在物理内存管理上，x86_64使用Multiboot信息获取可用物理内存区域，RISC-V解析FDT的/memory节点获取可用物理内存信息，在去除低地址区域和内核ELF占据的物理页后，物理页帧管理器将剩余可用物理内存按页划分，构建空闲页帧池，并提供统一的分配与回收接口。物理页帧回收工作可以借助Rust语言提供的RAII特性进行自动回收。内核链接脚本中预留了内核堆空间，这部分物理内存交给内核堆分配器进行分配。由于内核堆空间在内核ELF区域内，故不会与物理页帧分配器冲突。

在虚拟内存管理上，核心是定义一个跨架构的地址空间抽象，包含如下操作：获取页表根地址、创建/销毁地址空间、映射与取消物理页映射、查询虚拟地址到物理地址的映射。每种架构提供对应的页表项和页表寄存器实现：在riscv上，适配器实现Sv39语义并通过satp切换页表根；在x86_64上，适配器实现四级页表语义并通过CR3切换页表根。

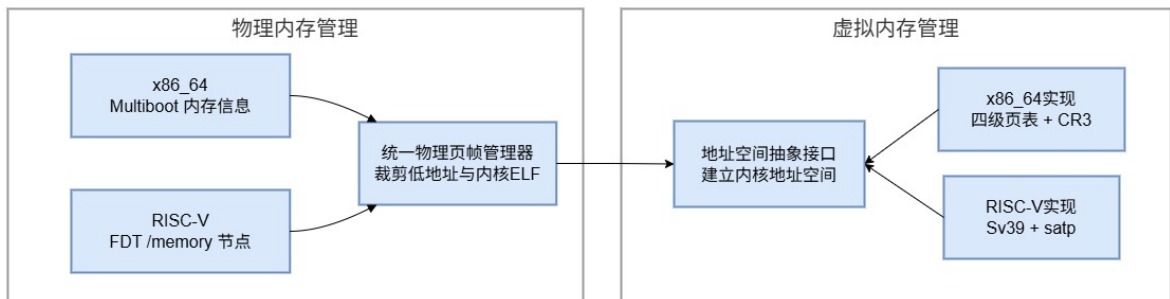


图 3-3 跨架构内存管理抽象设计

图3.3展示了内存管理部分的跨架构设计。物理内存管理通过Multiboot或FDT获取可用区域，经裁剪后交由统一的物理页帧分配器管理。虚拟内存管理定义地址空间抽象，x86_64与RISC-V分别实现四级页表与Sv39。上层业务逻辑仅依赖地址空间抽象，无需关心页表寄存器和层级差异，实现了内存管理的平台无关性。

3.1.3 中断控制

中断控制采用两层设计：汇编入口负责保存和恢复中断上下文、切换栈和地址空间，然后把控制权交给用Rust实现的中断分发函数。对于RISC-V，故障与系统调用共

享中断路径，仅需要区分内核中断与用户中断路径。x86_64引入了syscall指令和syscall MSR，需要额外区分故障和系统调用路径。故障处理时，部分寄存器的保存和恢复由CPU自动完成，剩余逻辑与RISC-V类似。系统调用则完全由软件保存和恢复所有寄存器，本文的做法是将percpu区域映射到用户态高地址，先暂存用户栈指针到内存中再恢复，以此完成换栈操作。中断分发函数负责解码异常原因、调用相应的处理逻辑并将结果写入用于恢复的中断上下文。这部分受架构影响较大，分别实现更好。

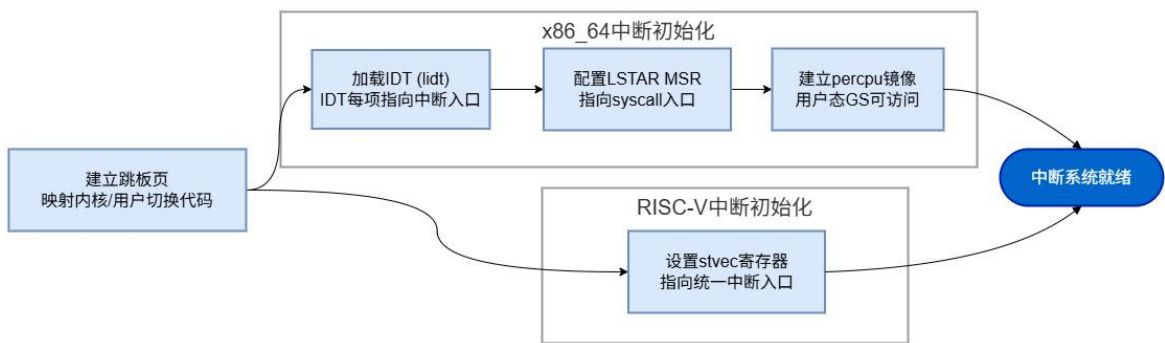


图 3-4 中断初始化双架构流程设计

图3-4展示了x86_64与RISC-V在中断初始化阶段的差异及共同收敛点。x86_64侧需要依次完成IDT汇编入口编写与加载、跳板页建立、syscall相关MSR配置以及percpu高地址镜像构建，步骤较多且涉及硬件特性。RISC-V侧仅需将stvec寄存器指向统一中断入口即可完成初始化。两架构最终均将控制权导向Rust中断分发函数，实现异常解码与处理的统一。该设计将平台相关的初始化细节封装在各自路径中，上层逻辑无需区分架构即可获取一致的中断服务能力。

3.1.4 设备探测与域加载

设备探测职责在于把平台发现设备的细节隐藏到平台实现中，对外只暴露出探测得到的设备信息。实现上，RISC-V使用FDT进行设备解析，x86_64则使用ACPI/PCI机制。不同平台存在不同平台的专有设备，例如PLIC、Goldfish等，即使是相同设备也存在不同的I/O方式，例如MMIO和PIO。在本文的设计中，对于平台专有设备的驱动域，假定在指定架构下实现，其它架构尝试编译会报错；对于平台共用设备的驱动域，其实现中支持多种I/O方式，域加载时需要按架构初始化I/O方式，这样就能隐藏

架构差异，复用同一段业务逻辑代码。上述两类域需要判断当前架构后加载。对于内核组件域中架构相关的部分，在实现中统一域接口操作，内部按架构实现。除此之外，还存在大量架构无关的内核组件域和虚拟设备域，这些域可以直接加载。由此便完成了设备探测与域加载的平台无关上层抽象。

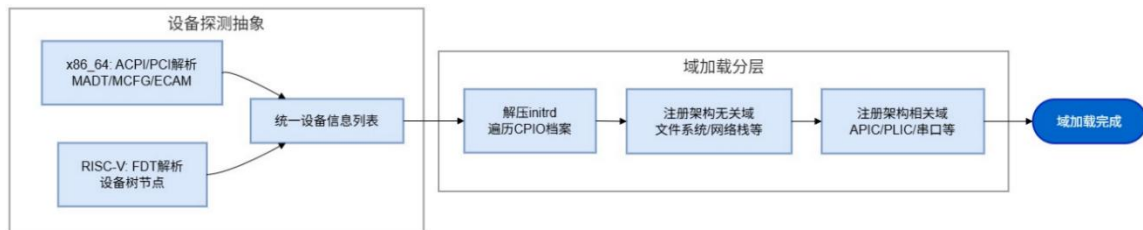


图 3-5 设备探测与域加载跨架构设计

图3-5展示了设备探测与域加载的跨架构设计。设备探测层将x86_64的ACPI/PCI解析和RISC-V的FDT解析封装为统一设备信息接口，屏蔽底层差异。域加载层先解压initrd并遍历CPIO档案，优先注册架构无关的通用域（如文件系统、网络栈等），随后根据各架构定义的列表注册平台特有域（如APIC、PLIC、串口驱动）。探测得到的设备信息用于指导域加载时的I/O方式初始化。该设计将架构差异收敛至域列表层面，通用逻辑实现代码复用。

总体上，引入x86_64的关键在于把架构专有步骤封装在少数几个边界：引导和早期初始化、地址空间抽象内部实现、汇编中断入口和中断分发函数、设备探测实现与域加载架构专有路径。只要这些边界的约定清晰且稳定，上层业务逻辑便无需了解底层的寄存器、页表或异常细节，从而实现跨架构的代码复用与可维护性。

3.2 vDSO 设计

本小节分为两部分：首先说明vdso_crate_template的设计目标与生成器职责，然后论述vDSO与syscall的设计与回退语义。

3.2.1 vdso_crate_template 设计

vdso_crate_template^[8]的核心职责是生成符合vDSO^[14]要求的动态链接库，并提供内核和用户库的加载和访问接口。它不直接承担具体的物理内存和用户页表管理，而是把这些职责抽象为接口，由内核去完成。

在vds0_crate_template的设计中，数据分为共享数据和私有数据，如图3-6所示。vDSO代码段和vVAR共享数据区映射到相同的物理页，被所有进程共享，设置为只读，从而保证安全且无需为每个进程复制。私有数据区为每个进程独立分配物理页，仅本进程可读写，用于存储与进程相关的状态。该布局通过页表将共享物理页映射到不同进程的虚拟地址，实现高效的内存复用与数据隔离。

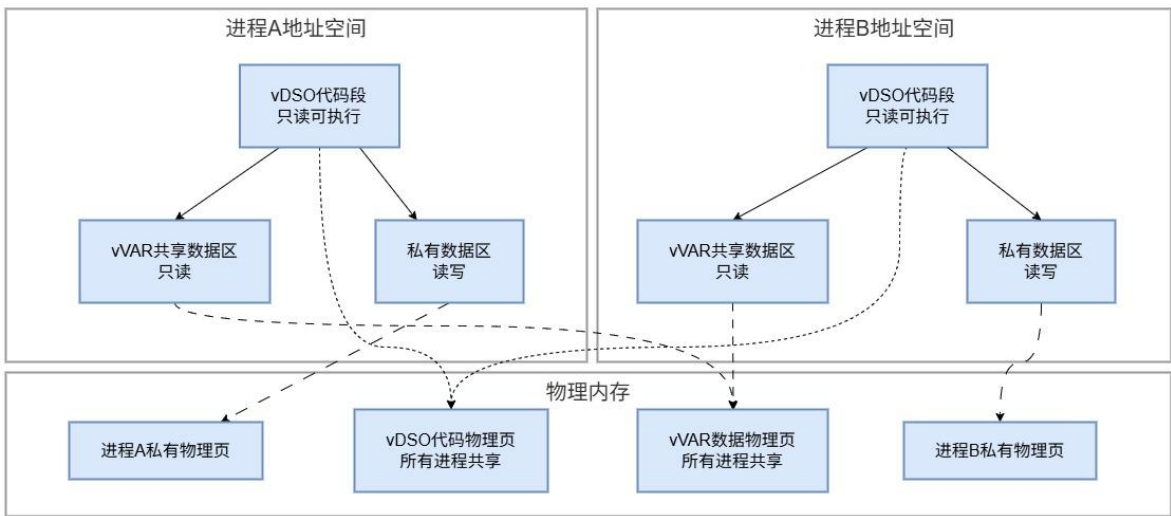


图 3-6 vDSO 共享数据与私有数据布局

生成器在构建时输出两类主要结果，即vDSO共享对象和静态API库。内核可以通过API库自动处理vDSO代码和数据的加载和重定位。用户库，可以通过API库像普通函数调用一样调用vDSO函数，因为API库记录了函数API及其在vDSO的偏移，只要用户提供了auxv提供的vDSO起始地址，就可以计算出vDSO函数地址。

在实际映射流程中，生成器通过一个内存接口由内核完成地址与物理页的分配与映射。这个接口把vDSO加载过程抽象出分配虚拟地址、分配物理页、页映射、权限更改和虚拟地址翻译这几个内核实现，使生成器逻辑能够同时适配内核初始化与映射到用户地址空间两种场景。

MemIf接口与功能对应如下：

表 3-1 MemIf 接口设计

接口名	功能说明
MemIf::valloc	在指定地址空间中预留一段虚拟地址区域供 vVAR 与 vDSO 使用，不立即分配物理页，返回该区域的基址。

MemIf::ppage_alloc	分配连续的或若干物理页用于存放 vVAR 或 vDSO 数据，返回物理页句柄。
MemIf::ppage_clone	复制物理页句柄以便在不同地址空间中安全地复用相同物理页，或按实现进行引用计数处理。
MemIf::map	将指定的物理页句柄映射到目标地址空间的虚地址，并设置只读、可执行、用户可见等权限标志。
MemIf::change_protect	在映射完成后修改该虚地址区间的权限，用于把 vVAR 先以可写方式初始化后切换为只读，或把代码段设为只读可执行。
MemIf::get_kernel_vaddr	在内核上下文中获取映射到用户地址的内核可访问虚拟地址，便于内核在初始化阶段直接向 vVAR 写入数据或验证映射内容。

内核端在首次加载时完成完整的ELF解析、段拷贝和重定位，在后续为其他地址空间建立映射时可以复用已加载的物理页。为了兼容物理页句柄通过Drop trait自动释放的设计，MemIf还提供了ppage_clone用于延续物理页生命周期。

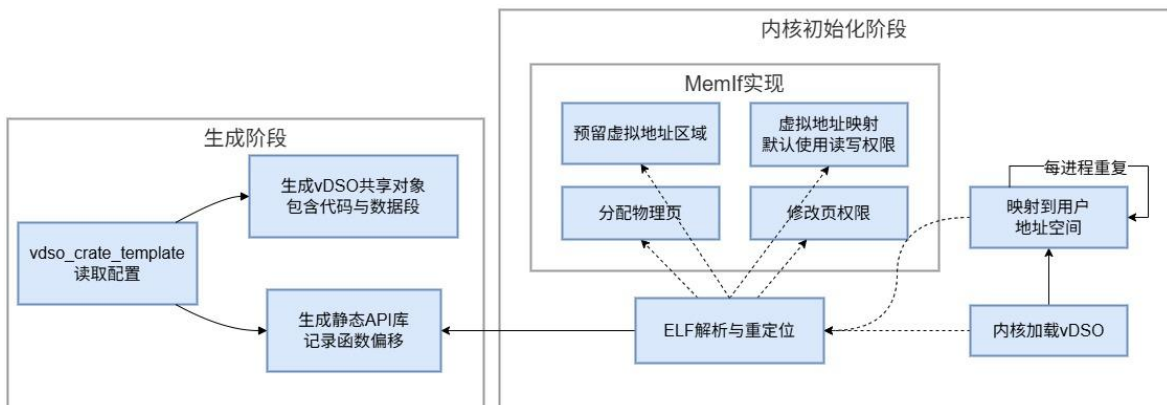


图 3-7 vDSO 生成与映射流程

vdsso_crate_template在生成阶段输出vDSO共享对象和静态API库，前者包含代码与数据段，后者记录各函数的偏移。内核初始化阶段通过MemIf接口顺序完成虚拟地址预留、物理页分配、段映射和权限设置，并在首次加载时执行ELF解析与重定位。后续进程可复用已加载的物理页，仅需重新建立虚拟映射。

3.2.2 vDSO 与 syscall 设计

本文为测试vDSO功能，在vDSO中设计了clock_gettime和gettimeofday的快速路径实现。这两个函数不存在私有状态，全部状态均存在于共享区。

在`clock_gettime`的总体设计中，vDSO内部只负责从共享区中读取当前时间并把结果转换为用户态所需的格式，不参与时间维护。

`gettimeofday`与`clock_gettime`共享同一套时间快照，实现逻辑类似，都是只从共享区内读取时间值，转换为对应的时间格式。这也是实现这两个系统调用的原因。

内核侧的刷新逻辑负责维护这份共享时间快照。如图3-8所示，内核在时钟中断读取当前时间，再把最新值写入共享区，并通过原子变量标记告知用户是否正在更新。用户态读取时先检查更新是否完成，再读取时间并在必要时重试，从而避免读到更新中的脏数据。由此，`clock_gettime`和`gettimeofday`在正常情况下都可以直接命中vDSO快路径，而当共享区暂不可用、内核正在更新中或其它异常情况时，统一回退到`syscall`，保证语义完整和运行稳定。

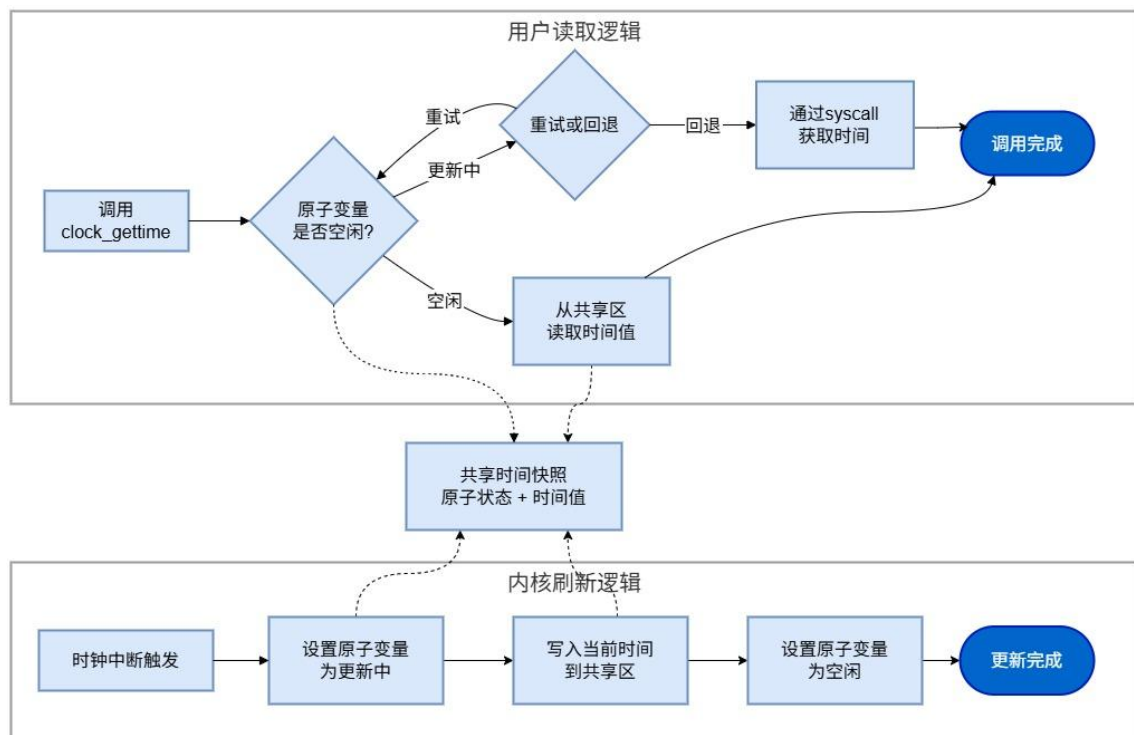


图 3-8 内核时间刷新与用户读取同步机制

内核在进程初始化时先建立vDSO和共享时间区域的映射，再通过`auxv`把vDSO入口暴露给用户态解析，用户态进行缓存。随后内核持续刷新共享时间快照，使用户态始终能够通过同一套vDSO接口获得较低开销的时间查询结果。

第4章 x86_64 移植与实现细节

本章描述将系统从RISC-V迁移到x86_64时的实现要点，聚焦启动与早期初始化、percpu与早期初始化、页表与寄存器差异、trap/syscall的切换语义、以及ACPI/PCI的探测方法。

4.1 引导与早期初始化

为了使用Multiboot进行引导，首先按照Multiboot1规范^[12]，通过链接脚本控制在二进制文件头部嵌入一个Multiboot header，指定ELF加载范围，以及入口点。x86_64的早期启动需启用分页模式，加载早期页表，完成控制寄存器与长模式相关配置，最后ljmp进入64位代码。

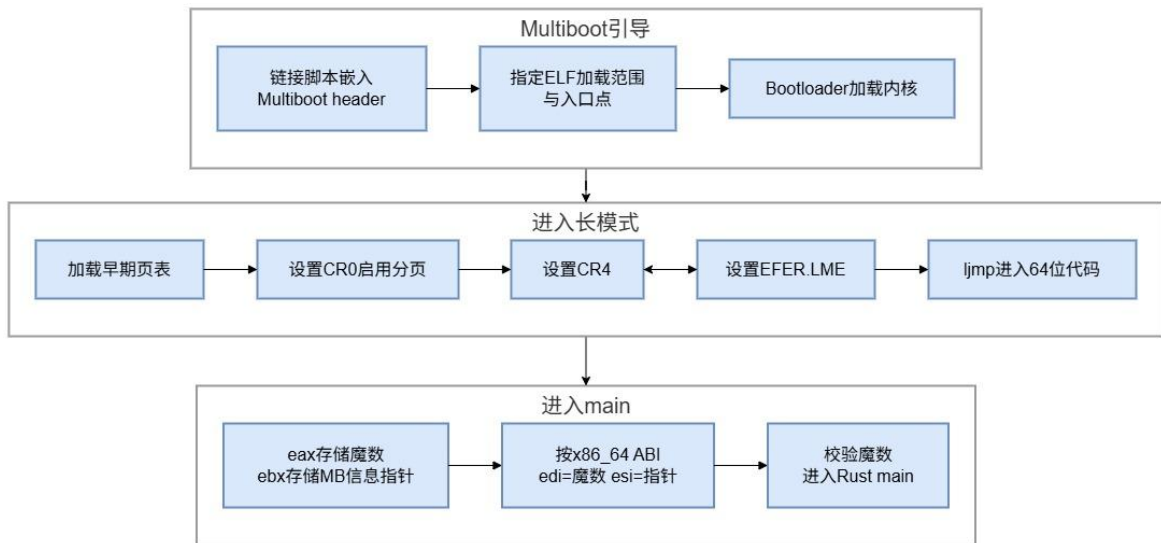


图 4-1 x86_64 Multiboot 至 main 启动流程

Multiboot引导完成后，eax中存储着魔数，ebx中是Multiboot信息指针。根据x86_64 ABI，将魔数作为第1个参数移到edi，Multiboot指针作为第2个参数移到esi后，就可以调用Rust函数，校验魔数后进入Rust编写的主函数。

第一步是初始化percpu寄存器和cpuid。引入percpu crate需要在链接脚本中添加一个percpu段。percpu的初始化在预先分配的内存中构造每核数据区并设置percpu寄存器值，以便在陷阱/系统调用时通过GS快速访问。cpuid在RISC-V侧SBI引导后通过寄

寄存器传递给entry，而x86_64侧可以通过封装底层cpuid汇编操作的raw_cpuid crate获取cpuid，然后写入percpu区。自此之后，所有的cpuid获取都通过percpu区获取。

表 4-1 早期初始化总流程

```
Procedure early_init()
    // 输入：无
    // 输出：无，完成各架构早期初始化

    percpu_area ← alloc_percpu_memory()
    cpuid_val ← raw_cpuid()
    percpu_area.set_cpuid(cpuid_val)
    set_percpu_register(percpu_area.base)

    If arch = x86_64 Then
        init_fpu_sse()
        init_primary_apic()
    End

    init_early_timer()

End early_init
```

接下来是平台早期初始化。对于x86_64，每个CPU在进入内核运行之前需要初始化FPU/SSE^[6]支持并配置本地APIC功能。FPU/SSE初始化的操作首先在CR0上清除FPU模拟位并设置MP，允许协处理器按x86语义工作；随后通过CR4启用OSFXSR与OSXMMEXCPT，从而支持使用FXSAVE/FXRSTOR与SSE的异常模型。执行fninit将x87状态复位为已知值，保证后续浮点指令在已定义的状态下运行。

主核APIC初始化首先通过关闭旧式8259A PIC避免中断冲突，随后检测CPU是否支持x2APIC，并将结果写入全局变量IS_X2APIC。根据是否启用x2APIC决定xapic_base，x2APIC模式通过MSR访问无需MMIO基地址，否则获取物理地址并转换为内核虚拟地址。用该基址和模式创建LocalApicContext，调用enable对硬件做寄存器编程，例如设置timer vector、配置timer等。最后将上下文写入互斥锁保护的全局变量并通过原子变量更新就绪状态。在x2apic crate^[5]的设计中，LocalApic对象仅记录设置项，只有在调用enable或enable_timer时才会真正写入寄存器。因此，主核创建LocalApicContext对象后，从核可以直接在这个对象上调用enable，对自己核上

的APIC进行配置。RISC-V得益于其简洁的设计，无需进行FPU/SSE和APIC相关初始化工作，早期初始化可立即完成。

4.2 x86_64 与 RISC-V 的页表结构与寄存器差异

x86_64使用四级页表（PML4→PDPT→PD→PT），每级512项，索引宽度为9位。RISC-V的Sv39使用三级页表（PGD→PMD→PT），每级512项，索引宽度为9位。

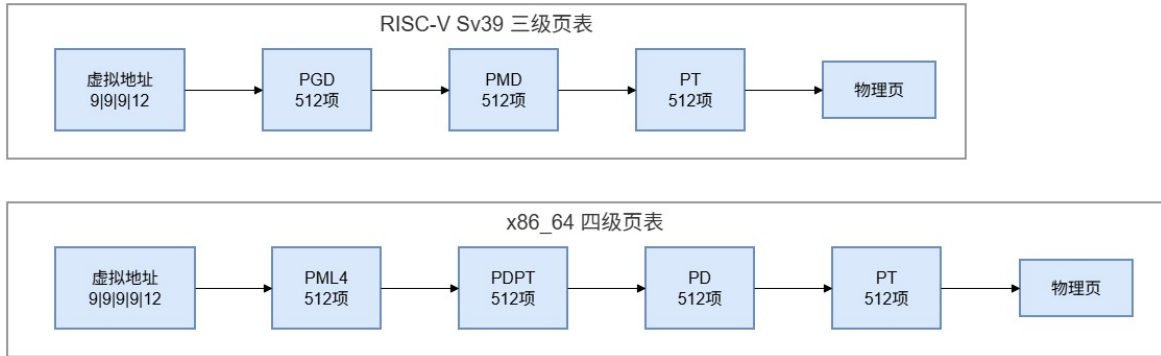


图 4-2 x86_64 与 RISC-V 页表层级对比

对于页表项，抽象出的主要操作包括：创建指向指定物理页的页表项、设置页表项权限、获取页表项指向的物理地址和权限。对于页表整体，抽象出的主要操作包括：映射虚拟页到物理页、修改映射的物理页或权限、删除映射、获取页表根物理地址、查询虚拟地址映射的物理地址。这些操作被统一封装为VSpace抽象接口，其核心原语如表4-2所示。

表 4-2 VSpace 抽象接口与页表激活流程

```
Interface VSpace
    // 获取页表根物理地址
    Function root_paddr() → PhysAddr
    // 映射虚拟页到物理页，可指定权限
    Procedure map(vaddr: VirtAddr, paddr: PhysAddr, flags:
    PageFlags)
    // 取消映射
    Procedure unmap(vaddr: VirtAddr)
    // 查询虚拟地址对应的物理地址和权限
    Function query(vaddr: VirtAddr) → (PhysAddr, PageFlags)
    // 修改已有映射的权限
```

```

    Procedure protect(vaddr: VirtAddr, flags: PageFlags)
End Interface

Function kernel_page_table_token() → usize
    If arch = x86_64 Then
        Return kernel_page_table_root_paddr()
    Else If arch = riscv64 Then
        token ← (8usize << 60) |
(kernel_page_table_root_paddr() >> FRAME_BITS)
        Return token
    End
End Function

Procedure activate_paging_mode(token: usize)
    If arch = x86_64 Then
        Write CR3 with PhysFrame containing token
    Else If arch = riscv64 Then
        sfence_vma_all()
        Write satp with token
        sfence_vma_all()
    End
End Procedure

// 上层统一调用方式
activate_paging_mode(kernel_page_table_token())

```

在page-table crate的设计下，不同架构依据上述接口提供对应的页表项类型（ArchPTE）和页表类型（ArchPageTable），并通过VmSpace与PagingIf接口实现基于RAII的物理页面分配和自动释放。当VmSpace对象触发drop时，其内部通过PagingIf::alloc分配的物理页对象也将自动drop，返还占用的物理内存。对于上层业务逻辑，只需调用activate_paging_mode(kernel_page_table_token())即可完成页表切换，无需关注CR3与satp的底层差异。

4.3 trap/syscall 初始化

在x86_64中，中断初始化需要设置trap和syscall两条路径。由于Alien在RISC-V中的实现就是使用跳板页完成内核和用户地址空间的切换，本文在x86_64下也保留类似的实现。x86_64中断初始化的主要步骤如图4-3所示。

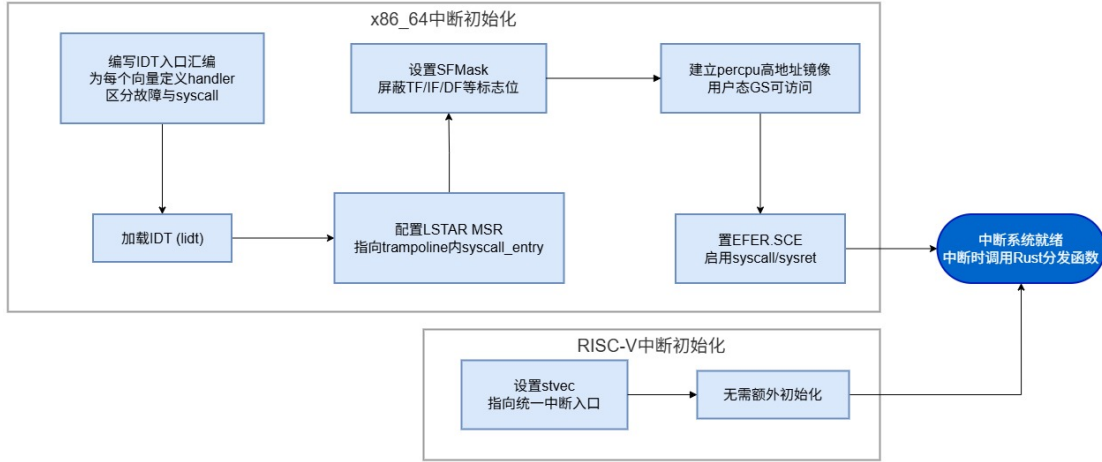


图 4-3 x86_64 与 RISC-V 中断初始化流程

上述初始化流程的关键部分可概括为IDT注册和syscall MSR配置，如表4-3所示。

表 4-3 IDT 与 syscall MSR 配置流程

```

Procedure init_interrupt_system()
    // 1. 设置 IDT: 为每个中断向量注册入口
    For vector ← 0 To 255 Do
        handler_addr ← trampoline +
        trap_handler_offset(vector)
        idt[vector] ← make_gate(handler_addr, DPL, IST)
    End
    lidt(idt)

    // 2. 配置 syscall 相关 MSR
    gs_value ← read_percpu_reg()
    gs_mirror ← PERCPU_MIRROR_BASE - _percpu_start + gs_value
    write_percpu_reg(gs_mirror)

    lstar_offset ← syscall_entry - strampoline
    lstar ← TRAMPOLINE + lstar_offset
    write_msr(LSTAR, lstar)

    sfmask ← RFlags.TRAP_FLAG | RFlags.INTERRUPT_FLAG |
    RFlags.DIRECTION_FLAG |
    RFlags.IOPL_LOW | RFlags.IOPL_HIGH |
    RFlags.NESTED_TASK | RFlags.ALIGNMENT_CHECK
  
```

```

write_msr(SFmask, sfmask)

write_msr(EFER, read_msr(EFER) | EFER_SCE)
End Procedure
    
```

完成硬件初始化后，`trap`与`syscall`将通过统一的汇编入口接管。两者在硬件行为与软件处理上存在显著差异，图4-4对这两条路径进行了对比。

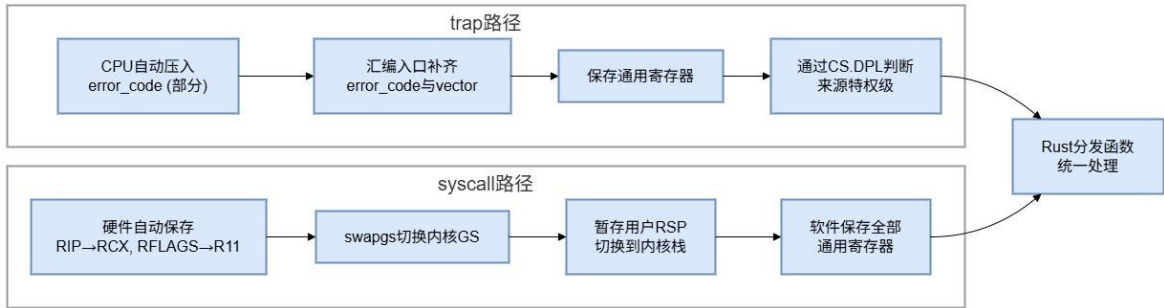


图 4-4 x86_64 trap 与 syscall 汇编入口路径对比

两条路径的分流逻辑如下：`trap`入口处，CPU对部分异常（8、10 - 14、17）自动压入错误码，对其它中断则压入0，以保证`trap`上下文格式统一。随后软件保存通用寄存器，并通过CS段选择子的DPL字段判断中断来源。若来自用户态（DPL=3），则执行`swapsgs`切换到内核GS基址，加载内核CR3和内核栈指针，完成地址空间切换；若来自内核态则直接复用当前栈和页表。

`syscall`路径则由硬件自动将RIP存入RCX、RFLAGS存入R11，且不压入错误码与中断向量。进入内核后首先通过`swapsgs`获取内核GS，再将用户栈指针暂存到`percpu`区域，借助TSS中的`rsp0`切换到内核栈。与`trap`路径不同，`syscall`必须由软件完整保存所有通用寄存器，因为硬件只保护了RCX和R11。返回时，将上下文中的RIP和RFLAGS分别恢复到RCX和R11，并直接从保存的用户栈指针恢复RSP，最后通过`sysretq`回到用户态。

值得强调的是`percpu`高地址镜像的设计：内核在初始化时将`percpu`数据映射到所有地址空间的高地址区域，并将`percpu`寄存器设置为该镜像基址。这样，无论当前运行在内核地址空间还是用户地址空间，GS始终指向同一份`percpu`数据，无需在地址空

间切换时修改GS，从而避免了额外的寄存器操作。该机制保证swaps仅需在进入和离开内核时各执行一次，即可使内核态正确访问percpu，用户态也能安全返回原有的GS基址。

Rust分发函数接收到统一的trap上下文后，解码异常原因、调用相应处理逻辑，并将结果写回上下文，最终恢复寄存器并通过iretq或sysretq返回原执行流。整个中断处理流程通过汇编入口层的架构专有处理与Rust分发层的逻辑分离，实现了平台差异的有效屏蔽。

4.4 ACPI 与 PCI 的探测方式与域加载

本文在x86_64上以ACPI^[10]作为设备发现的入口，开发中使用Rust语言编写的acpi crate^[4]。先读取RSDP，再解析SDT中的各张表，整理出平台信息。MADT提供中断控制器信息，MCFG提供PCI配置空间信息，SPCR提供串口信息，HPET提供定时器信息，FADT则补充电源和平台基础信息。这一步对x86_64很重要，固件给出的内容往往决定了后续设备能不能被正确识别。本文不假定固定硬件布局，尽量按ACPI表来判断。ACPI解析与PCI探测的总体流程如图4-5所示。

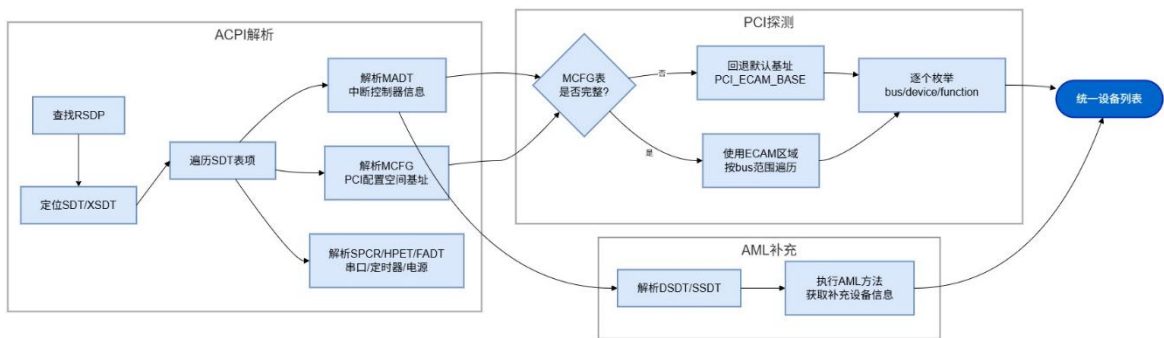


图 4-5 ACPI 解析与 PCI 探测流程

在PCI^[11]探测上，优先使用MCFG。MCFG给出ECAM区域后，内核按bus范围访问PCI配置空间，逐个找出总线上的设备。如果ACPI表完整，PCI探测就按表中的信息走；如果表缺失或不完整，再退回到平台默认的PCI_ECAM_BASE作为兜底。这样做的目标是保证在标准固件上按标准方式工作，信息不足时也还能启动，不会因为一处表缺失就让整条设备链路中断。AML的作用主要是补足ACPI静态表里没直接写明的信息。某些平台把串口、总线或其他设备信息放在AML方法里，本文在需要时读

取这些方法，拿到更完整的设备描述，再把结果纳入统一的设备列表。实现时 APCI/AML 和 PCI 探测中都优先从探测中获取结果，失败后再回退到默认值并在日志中警告回退行为。上述探测逻辑可概括为表 4-4。

表 4-4 ACPI 与 PCI 设备探测流程

```
Procedure probe_devices() → DeviceList
    devices ← empty_list
    rsdp ← find_rsdp()
    sdt ← parse_sdt(rsdp)

    For each table in sdt Do
        If table.type = "MADT" Then
            ioapic_list ← parse_madt(table)
            devices.add(ioapic_list)
        Else If table.type = "MCFG" Then
            ecam_list ← parse_mcfg(table)
        Else If table.type = "HPET" Then
            hpet_info ← parse_hpet(table)
            devices.add(hpet_info)
        End
    End

    If ecam_list is not empty Then
        For each ecam in ecam_list Do
            devices.add(probe_pci_ecam(ecam))
        End
    Else
        devices.add(probe_pci_legacy(PCI_ECAM_BASE))
    End

    aml_devices ← parse_aml(sdt.dsdt, sdt.ssdt)
    devices.add(aml_devices)

    Return devices
End Procedure
```

本文在域加载阶段按预先定义的列表，逐个加载内核所需的驱动和服务。该过程分为两层：架构无关层处理所有平台共用的加载逻辑，架构相关层定义各平台特有的域。

架构无关层在`init_domains()`中实现。本文从`initrd`中读取`gzip`压缩数据，解压后遍历CPIO档案内容，查找域ELF二进制文件，再依次调用`register_domain_elf`加载并注册。本文先注册`COMMON_INIT_DOMAIN_LIST`中定义的通用域，包括缓冲UART、缓冲输入、块设备缓存、设备文件系统、FAT文件系统、管道文件系统、进程文件系统、内存文件系统、系统文件系统、调度器、任务管理、虚拟文件系统和网络栈等。这些域与硬件平台无关，在所有架构上代码和逻辑保持一致。

架构相关层由各架构的`ARCH_INIT_DOMAIN_LIST`定义。`x86_64`上的列表包括`uart16550`串口、本地APIC域、IO APIC域、CMOS RTC、`virtio`设备等，这些域对应`x86_64`特有的中断控制器和设备驱动。`riscv64`上的`ARCH_INIT_DOMAIN_LIST`则包括`goldfish` RTC、PLIC中断控制器、QEMU上的`uart16550`和`virtio`设备，以及VisionFive 2平台上的`uart8250`和SD卡控制器，反映出架构的硬件差异。`init_domains()`在注册完通用域后，随即遍历`ARCH_INIT_DOMAIN_LIST`，同样调用`register_domain_elf`注册这些架构特有的域。这种分层设计确保通用部分在所有架构上只维护一份代码，各架构可在域列表层面灵活定制所需的驱动和服务，避免在通用加载逻辑里散布平台判断。

表 4-5 域加载流程

```
Procedure init_domains()
  archive ← decompress_gzip(initrd_data)
  For each entry in cpio_iter(archive) Do
    If entry.is_elf() Then
      domain ← register_domain_elf(entry)
    End
  End

  For each name in COMMON_INIT_DOMAIN_LIST Do
    register_domain_elf_by_name(name)
  End

  For each name in ARCH_INIT_DOMAIN_LIST Do
    register_domain_elf_by_name(name)
  End
End Procedure
```

结合设备探测结果与域加载机制，内核得以在x86_64平台上构建完整的驱动栈。探测阶段通过ACPI/PCI获取的设备信息为域加载提供依据，而域加载的分层设计使得架构相关驱动仅需在列表层面声明即可完成注册，无需修改通用逻辑。

第5章 vDSO 实现与用户态集成

本章介绍vDSO（virtual Dynamically linked Shared Object）在本项目中的实现思路与工程细节，重点讨论构建与产物、内核侧的映射与刷新机制、导出API与用户态解析。vDSO的整体生命周期跨越构建时、内核初始化时和内核与用户运行时三个阶段，如图5-1所示。

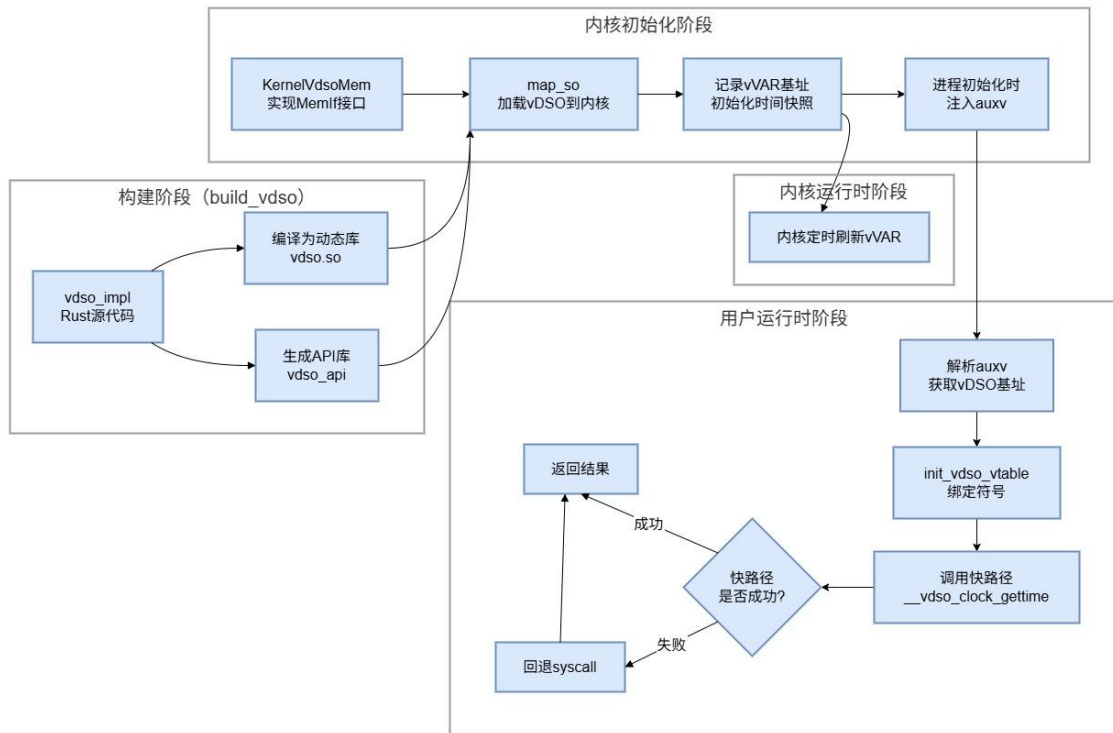


图 5-1 vDSO 构建、装载与调用总览

vDSO的完整流程始于构建阶段：vdso_crate_template将vdso_impl编译为可映射的共享对象和API库，API库记录了函数符号与偏移。内核初始化阶段，KernelVdsoMem实现MemIf接口，通过map_so完成vDSO的ELF解析、段拷贝和重定位，同时记录vVAR共享区基址以便后续刷新。进程创建时内核通过auxv将vDSO入口注入用户态。用户运行时阶段，userlib解析auxv获取基址，初始化函数跳转表，随后即可直接调用vDSO快路径；若调用失败则自动回退到syscall，保证功能完整。以下各节分别阐述构建、内核侧实现、刷新策略与用户态加载的细节。

5.1 vdso_crate_template 构建与产物

`vdso_crate_template`由`build_vdso`和`vdso_helper`两个子组件构成。`build_vdso`负责把`vdso_impl`编译成可映射的so和api库，`vdso_helper`负责在vDSO实现里生成vVAR访问宏与数据布局。`api`库提供了vDSO函数的ABI以省去手写胶水代码，以及供内核调用的加载与映射辅助函数。构建流程如图5-2所示。

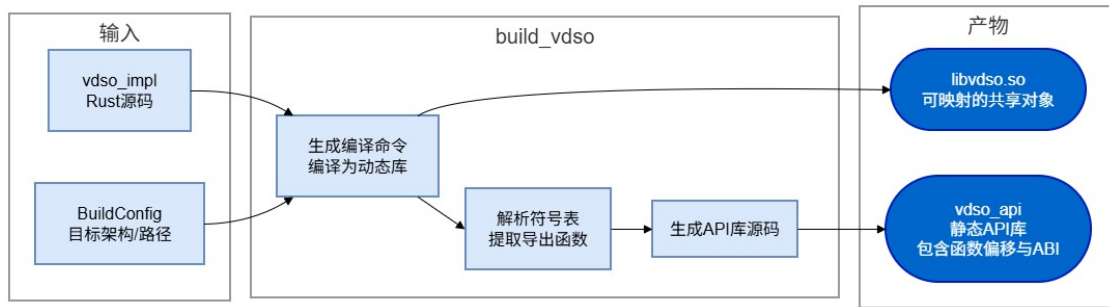


图 5-2 vDSO 构建流程

首先展示vDSO的构建过程，由项目中的`vdso_builder crate`完成。`vdso_builder`只需要提供目标架构、输入与输出路径、创建的so文件名称和api库名称等信息构建`BuildConfig`，然后调用`build_vdso::build_vdso()`即可。该过程可概括为表5-1。

表 5-1 vDSO 构建配置与调用

```

Procedure build_vdso_for_arch(target_arch: TargetArch,
repo_root: Path)
    config ← new BuildConfig
    config.arch ← target_arch.to_string()
    config.out_dir ← repo_root / "build/vdso"
    config.so_name ← "libvdso"
    config.api_lib_name ← "vdso_api"
    config.toolchain ← "nightly-2026-01-23"
    config.input_path ← repo_root / "vdso/vdso_impl"

    build_vdso_tool(config)
End Procedure
  
```

在`build_vdso`内部，首先根据输入的配置生成编译命令，将输入项目编译为动态链接库，然后通过解析输入项目的Rust源代码和动态链接库的函数符号生成api库。

vDSO函数实现由项目中的vdso_impl定义。根据build_vdso的设计，vdso_impl需要保留一个api模块用于导出公共符号，其余内部实现可以放到其它模块中。vdso_impl通过vdso_helper中的vvar_data!与get_vvar_data!声明共享快照区，并在api模块导出__vdso_clock_gettime等符号。共享快照区包含三个字段：seq（原子版本号）、realtime_ns和monotonic_ns，分别用于协调读写同步及存储两种时钟的纳秒值。用户态读取时间快照时采用无锁的seqcount机制，如表5-2所示。

表 5-2 用户态 seqcount 读取逻辑

```
Function read_clock_timespec(clock_id: usize) → Option<TimeSpec>
  If clock_id = CLOCK_REALTIME Then
    field ← "realtime_ns"
  Else If clock_id = CLOCK_MONOTONIC Then
    field ← "monotonic_ns"
  Else
    Return None
  End

  Loop
    seq ← read_volatile(vvar.seq)
    If (seq & 1) ≠ 0 Then
      hint_spin_loop()
      Continue
    End
    nanos ← read_volatile(vvar.field)
    seq_after ← read_volatile(vvar.seq)
    If seq = seq_after Then
      tv_sec ← nanos / 1_000_000_000
      tv_nsec ← nanos % 1_000_000_000
      Return Some(TimeSpec(tv_sec, tv_nsec))
    End
  End Loop
End Function
```

本文的实现基于奇偶版本号（seqcount）：内核作为写者，用户程序作为读者。读者先读取一次seq，若为奇数表示内核正在更新，则自旋等待后重试；若为偶数则读取目标时间字段，再读取一次seq。只有前后两次seq相等且均为同一偶数，才认为拿到了一组一致的快照，否则重试。代码中使用volatile读和core::hint::spin_loop，目

的是避免编译器优化或CPU重排序的干扰，强制重新读取值。对于clock_gettime这类高频读取场景，这种模式能显著降低延迟。如果多次重试仍拿不到一致快照，调用方可以退回到系统调用，由内核保证正确结果。

__vdso_clock_gettime作为导出函数，在检查用户指针非空后调用上述读取逻辑，成功则将TimeSpec写入用户缓冲区并返回0，失败则返回-1。该函数在vDSO中始终只读取共享区，不参与时间维护，这使其实现足够轻量且可安全地在用户态高频执行。

5.2 内核侧 MemIf 实现与 vDSO 装载

vDSO引入需要内核侧MemIf实现来完成物理页分配、页表映射等工作。本文在KernelVdsoMem中实现了vdso_api::MemIf，并在init_vdso中调用vdso_api::map_so完成装载。装载流程及MemIf各接口的调用关系如图5-3所示。

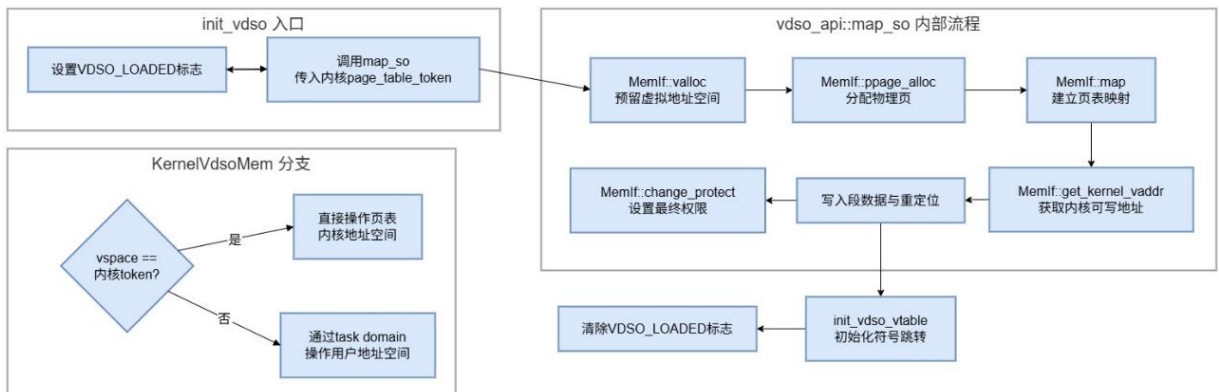


图 5-3 内核侧 MemIf 装载流程

内核首次加载vDSO与后续向用户地址空间映射均使用vdso_api::map_so，区别仅在于传入的vspace参数不同。调用时vspace传入页表token；MemIf内对所有带vspace参数的操作，先判断该token是否为内核页表token：若是则直接在本地实现页表操作；否则转而调用task domain提供的跨地址空间服务，从而保持接口统一。

物理页分配的核心是借用RAII实现自动回收。FrameTracker对象在Drop时自动释放物理页，但多个地址空间可能共享同一物理页，因此ppage_alloc将FrameTracker包装为Arc<FrameTracker>，再转为裸指针返回usize；ppage_clone则通过增加引用计数来延长生命周期。上述关键实现可概括为表5-3。

表 5-3 物理页管理及 vDSO 装载入口

```

// 物理页分配：返回引用计数句柄
Function ppage_alloc(size: usize) → PhysPagePtr
    frame ← FrameTracker::alloc(size)
    arc ← Arc::new(frame)
    raw_ptr ← Arc::into_raw(arc)
    Return raw_ptr as PhysPagePtr
End Function

// 物理页句柄克隆：增加引用计数
Function ppage_clone(ppage: PhysPagePtr) → PhysPagePtr
    arc ← unsafe { Arc::from_raw(ppage as *const FrameTracker) }
    cloned ← Arc::clone(&arc)
    // 避免原 Arc 被释放，再次转为原始指针
    let _ = Arc::into_raw(arc)
    Return Arc::into_raw(cloned) as PhysPagePtr
End Function

// 内核端 vDSO 装载入口
Procedure init_vdso()
    set_vdso_loaded_flag(true)
    vdso_base ← vdso_api::map_so(kernel_page_table_token())
    init_vdso_vtable(vdso_base)
    set_vdso_loaded_flag(false)
End Procedure

```

`map_so`内部借助`MemIf`依次完成虚拟地址预留、物理页分配、页表映射、内核临时可写映射建立、段数据拷贝与重定位，最后通过`change_protect`将代码段设为只读可执行、数据段设为只读。对于`vVAR`共享区，则先以可写方式初始化时间快照，再切换为只读。一旦装载完成，内核记录`vVAR`基址供后续刷新使用，同时通过`auxv`将`vDSO`基址暴露给用户态，使用户运行时能够直接绑定`vDSO`符号并调用快路径。

5.3 内核侧刷新策略

内核在首次加载`vDSO`时记录了`vVAR`共享区的基址，随后利用该地址在每次时钟中断或其他定时事件中刷新时间快照。刷新过程采用与用户态读取对应的`seqcount`写序列：先将版本号`seq`置为奇数表示“写入中”，然后更新各时间字段并通过内存

屏障保证写入顺序，最后将seq置为偶数表示“写入完成”。整个刷新流程如图5-4所示。

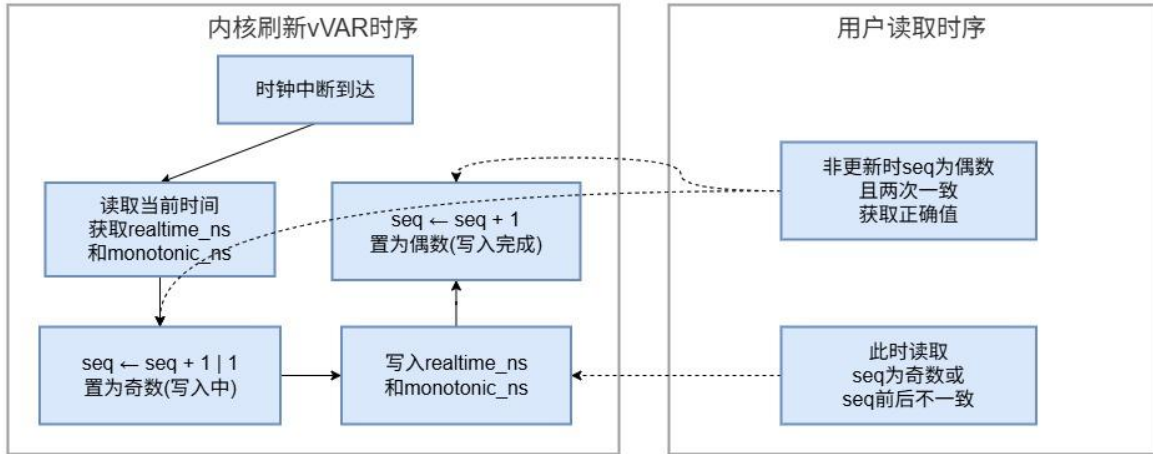


图 5-4 内核刷新 vVAR 与用户读取同步

对应的刷新逻辑如表5-4所示。

表 5-4 内核 vVAR 刷新流程

```

Procedure refresh_vvar()
  data ← unsafe { &mut *(vvar_base as *mut VvarData) }
  realtime_ns ← read_current_realtime_ns()
  monotonic_ns ← read_current_monotonic_ns()

  // 写序列开始：将 seq 置为奇数
  seq ← data.seq.wrapping_add(1) | 1
  data.seq ← seq
  compiler_fence(Ordering::Release)

  // 写入时间字段
  data.realtime_ns ← realtime_ns
  data.monotonic_ns ← monotonic_ns
  compiler_fence(Ordering::Release)

  // 写序列结束：将 seq 置为偶数
  data.seq ← seq.wrapping_add(1)
End Procedure
    
```

这种无锁的seqcount写序列仅通过版本号的奇偶变化协调读写，用户态读取时无需内核参与即可判断数据一致性。内核写操作不阻塞读者，读者在偶数列看到一致数据，奇数时自旋等待，从而在保证正确性的同时将读路径延迟降至最低。

5.4 用户运行时 vDSO 加载

本文把用户态vDSO的加载逻辑集中放在用户运行时(userlib)里。userlib通过解析程序初始栈上的auxv获取vDSO ELF的基址，然后初始化vdso_api的vtable。之后调用方就能直接使用vDSO导出的快速路径接口（例如__vdso_clock_gettime）。当vDSO不可用或调用失败时，userlib会退回到传统的syscall实现。该过程如图5-5所示。

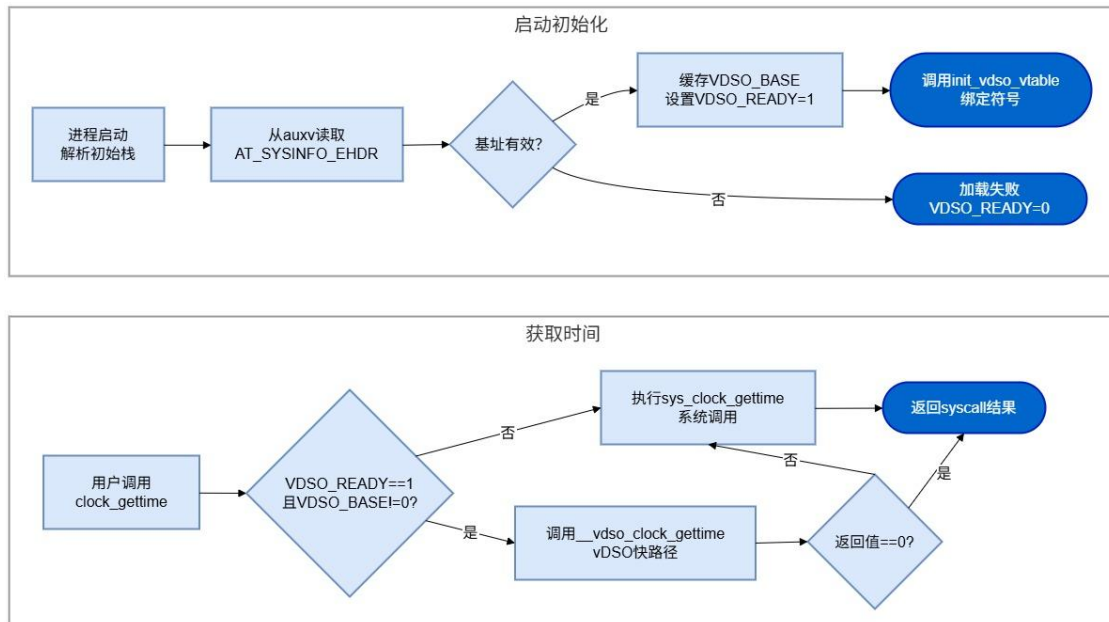


图 5-5 用户运行时 vDSO 初始化与调用流程

对应的实现逻辑由表5-5给出。

表 5-5 用户运行时 vDSO 访问与回退

```

// 全局原子变量
VDSO_BASE ← 0
VDSO_READY ← 0

Procedure init_from_stack(argc_ptr: usize)
    base ← parse_vdso_base(argc_ptr)
    
```

```
    If base is valid Then
        VDSO_BASE.store(base, Ordering::Release)
        unsafe init_vdso_vtable(base)
        VDSO_READY.store(1, Ordering::Release)
    End
End Procedure

Function clock_gettime(clk: usize, ts: &mut TimeSpec) → bool
    If VDSO_READY.load(Acquire) != 0 and VDSO_BASE.load(Acquire) != 0 Then
        ret ← unsafe __vdso_clock_gettime(clk, ts as *mut _)
        If ret == 0 Then
            Return true
        End
    End
    // 回退到系统调用
    Return sys_clock_gettime(clk, ts) == 0
End Function
```

在进程启动时userlib从用户栈上解析AT_SYSINFO_EHDR并缓存vDSO基址，然后用vdso_api::init_vdso_vtable()把vDSO导出的符号绑定到用户态可调用的函数指针或跳转表。当快路径调用失败时，回退到sys_clock_gettime系统调用，保证功能完整。

第6章 验证与评估

本章基于一次完整运行的实验日志，系统地说明将系统迁移到x86_64平台后在功能性和性能方面的验证结果。本章依次讨论测试设计、功能验证、性能度量，结论部分总结主要发现和后续工作方向。

6.1 测试设计

在测量vDSO快路径相对于传统syscall路径的性能之前，本文首先通过功能测试确认两者返回的时间值一致。测试流程如图6-1所示：先执行16轮功能验证，每轮依次调用vDSO与raw syscall获取单调时钟，将结果转换为纳秒并计算绝对差值；若任一差值超过阈值(1ms)则判定失败，确保进入性能测试前两条路径均能返回正确时间。

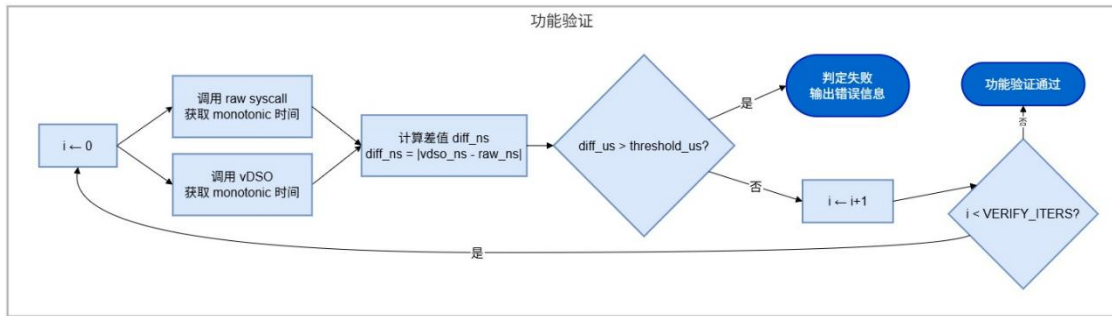


图 6-1 功能验证流程

功能验证的核心逻辑可概括为表6-1。

表 6-1 vDSO 功能验证逻辑

```

Function verify_correctness(threshold_us: u64) → Result
  For i ← 0 To VERIFY_ITERS - 1 Do
    vdso_ret ← clock_gettime_vdso(CLOCK_MONOTONIC, &vdso_ts)
    raw_ret ← clock_gettime_raw(CLOCK_MONOTONIC, &raw_ts)
    Assert vdso_ret = 0 and raw_ret = 0

    vdso_ns ← timespec_to_ns(vdso_ts)
    raw_ns ← timespec_to_ns(raw_ts)
    diff_ns ← |vdso_ns - raw_ns|
    diff_us ← (diff_ns + 999) / 1000
  
```

```
If diff_us > threshold_us Then
    Print "iter=" i "vdso_ns=" vdso_ns "raw_ns=" raw_ns "diff_us="
diff_us
    Return Err("vdso vs raw difference exceeds threshold")
End
End
Return Ok
End Function
```

性能测试采用窗口测量法，以避免计时操作本身干扰被测调用。其原理如图6-2所示：在窗口开启前后各通过一次系统调用获取单调时钟作为起止时间，窗口内连续执行大量被测调用，用总耗时除以调用次数得到单次平均耗时。这种方法将系统调用的固定开销均摊到所有样本上，有效减小计时偏差。

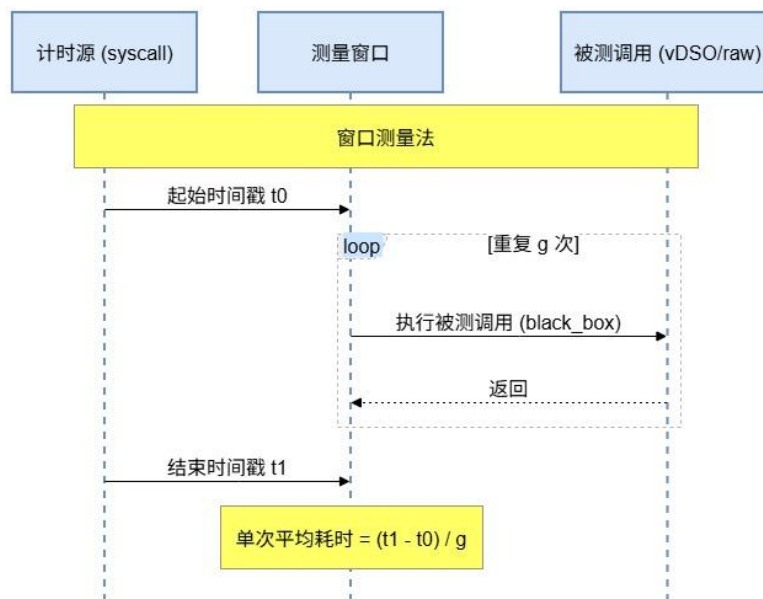


图 6-2 窗口测量法时序示意

为了让缓存不干扰结果，每组正式测量前都先跑64次预热，让指令和数据尽可能进缓存，也让分支预测等硬件状态稳定下来。同时，为了避免编译器把测量循环优化掉（比如觉得循环里没什么用就直接删掉），测量时用`core::hint::black_box`把被测调用包起来，强制编译器保留调用，保证真的执行了那么多遍。

调用次数选了64、128、256、512、1024这几档，从少到多都覆盖到，方便看窗口变大后单次平均耗时会不会更稳定，以及不同调用次数下有没有缓存效应或别的波动。每组实验独立重复5次，然后算平均值和标准差，这样数据相对稳定。

表 6-2 窗口测量与统计流程

```

Procedure measure_groups(groups: List[usize]) → List[PerfResult]
    results ← empty_list
    For each g in groups Do
        // 预热
        For w ← 1 To WARMUP_ITERS Do
            black_box(clock_gettime_vdso(CLOCK_MONOTONIC, &ts))
            black_box(clock_gettime_raw(CLOCK_MONOTONIC, &ts))
        End

        vdso_avgs ← empty_list
        raw_avgs ← empty_list

        For rep ← 1 To REPEATS Do
            vdso_elapsed ← measure_window(g, VDSO_PATH)
            raw_elapsed ← measure_window(g, RAW_PATH)
            vdso_avg ← vdso_elapsed / g
            raw_avg ← raw_elapsed / g
            speedup ← raw_avg / vdso_avg

            vdso_avgs.push(vdso_avg)
            raw_avgs.push(raw_avg)
        End

        vdso_mean, vdso_std ← compute_stats(vdso_avgs)
        raw_mean, raw_std ← compute_stats(raw_avgs)
        agg_speedup ← raw_mean / vdso_mean

        results.push(PerfResult(samples = g, vdso_avg = vdso_mean, raw_avg =
raw_mean, ...))
    End
    Return results
End Function

Function measure_window(samples: usize, path: PathType) → u128
    t0 ← monotonic_now_ns()

```

```
For i ← 0 To samples - 1 Do
    If path = VDSO Then
        black_box(clock_gettime_vdso(CLOCK_MONOTONIC, &ts))
    Else
        black_box(clock_gettime_raw(CLOCK_MONOTONIC, &ts))
    End
End
t1 ← monotonic_now_ns()
Return t1 - t0
End Function
```

计时基准统一使用`monotonic_now_ns`，它只做一件事：调用`raw syscall`读取单调时钟，再把结果换算成纳秒。它的作用是给窗口前后提供同一个时间基准，避免用被测调用自己的返回值来回算耗时。

表 6-3 计时基准

```
Function monotonic_now_ns() → ul28
    clock_gettime_raw(CLOCK_MONOTONIC, &ts)
    Return timespec_to_ns(ts)
End Function
```

整体上，这套测试分成两步。第一步先做功能检查，重点看`vDSO`和`raw syscall`的结果是不是在可接受范围内。第二步再做性能比较，重点看两条路径的单次平均耗时差多少。也就是说，`vDSO`与`raw syscall`时间差距首先被当作功能测试项处理，只有在功能确认通过后，才继续看性能数据。

本文把测量误差分成了两类。一类是统计误差，来自单次运行的样本数量不够；另一类是系统性误差，来自调度、后台干扰或者测量代码本身的开销。为了降低统计误差，本文对每个样本组都做多次完整运行，再算均值和方差。为了减小系统性误差，测试在单核上运行，除了测试之外后台没有其它进程，并且保持`QEMU`的配置参数不变。

6.2 功能验证与运行证据

功能验证基于系统从引导到进入用户态测试并成功呈现交互shell的日志序列。图6-3截取了具有代表性的运行日志，依次展示引导与早期初始化、域注册与加载、用户态测试三个阶段的连贯输出。

```
[0] [x86_boot] main_entry magic=0x2badb002
[0] Frame allocator init success
[0] Build kernel address space success
[0] ++++ setup interrupt +++
[0] ++++ setup interrupt done, enable:true +++
[0] <register domain>: scheduler, logger, fatfs, ramfs, devfs, procfs, sysfs, pipefs,
domainfs, vfs, task, syscall, uart16550, local_apic, io_apic, cmos_rtc, virtio_net
[0] Load domains done
[syscall_all] start
[sys_poll] PASS
[sys_fs] PASS
[sys_link] PASS
[sys_proc] PASS
[sys_random] PASS
[sys_time] PASS
[sys_vdso_time_compare] PASS
[syscall_all] PASS
[Init] all tests passed
Alien:/#
```

图 6-3 运行日志节选

本文认为，这些日志片段给出了整条链路的证据。启动阶段验证了引导、内核早期初始化、内存初始化、中断初始化正确完成。域注册与加载片段验证了系统模块化加载逻辑和依赖解析。用户态测试通过则验证了系统调用转发、用户态库和运行时环境工作的正确性。虽然日志不可能穷尽所有边界情况，但它的连贯性和顺序性已足够作为功能完整性的主要证据。

6.3 性能度量与分析

功能验证通过后，本文开始测量性能。对vDSO和raw syscall两种方式，分别在不同样本组（64、128、256、512、1024）上用窗口法测量平均单次耗时。下面列出运行日志中记录的原始测量数据，作为后续统计分析的基础。

表 6-4 vdso 与 syscall 性能对比表

样本数	重复次数	vDSO 平均(ns)	vDSO 标准差(ns)	raw 平均(ns)
64	5	1327	512	38860
128	5	511	64	35357
256	5	363	64	37914
512	5	282	16	36551

由于vDSO与raw syscall的单次耗时相差两个数量级，同图展示会压缩vDSO的数据细节，因此本文分别绘制vDSO与raw syscall的柱状图（图6-4、图6-5）以清晰呈现各自的变化趋势。

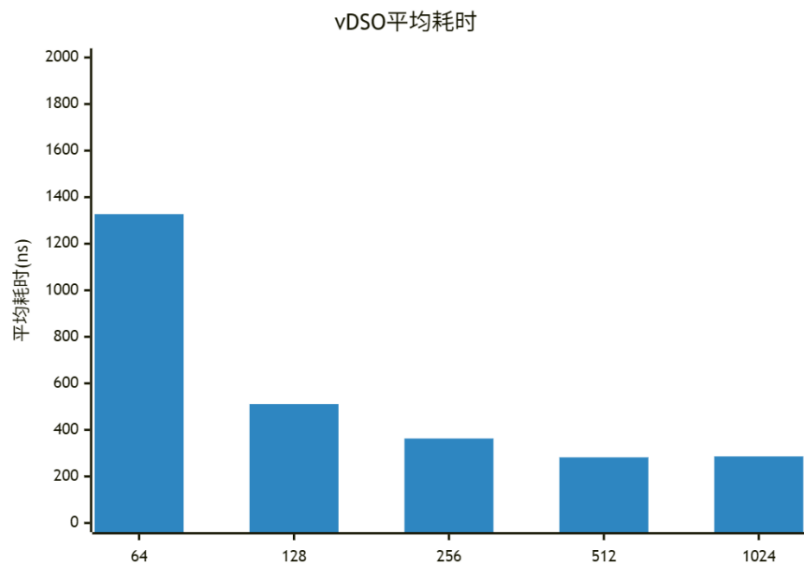


图 6-4 vDSO 平均耗时柱状图

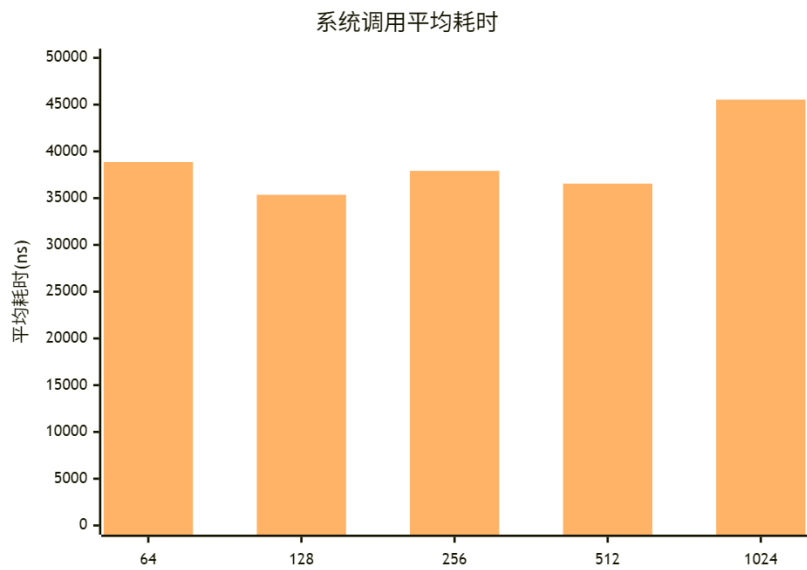


图 6-5 系统调用平均耗时柱状图

从图6-4可见，vDSO平均耗时随着样本数增加从1327 ns逐步下降，在512和1024样本组趋于稳定（282 ns和286 ns），表明缓存预热后vDSO路径的开销极低且基本稳定。图6-5显示raw syscall的平均耗时始终在35 000 ns至45 500 ns之间波动，未随样本数增大呈现明显的下降或收敛趋势，反映出系统调用路径的高开销特性。两组数据对比表明，vDSO在最佳情况下可获得超过两个数量级的加速比（以1024样本组为例，加速比约为159倍），同时其单次耗时标准差也更小，说明vDSO的性能表现更稳定。尽管vDSO自身开销极小，其测量值仍会受系统抖动和计时基准误差影响，因此后续工作将采用更大的样本数以进一步稀释误差，并尝试使用TSC等更轻量级的时间源以提升测量精度。

结 论

本文以工程实现为主，围绕将Alien系统从RISC-V平台迁移到x86_64并引入vDSO的设计展开设计和实现工作。在不破坏既有RISC-V路径的前提下，本文提出并实现了针对x86_64的平台抽象，完成了启动与早期初始化、内存管理、中断与syscall适配等关键子系统的移植工作，构建了域加载与设备探测的链路，并实现了vDSO/vVAR的引入以提供用户态的低延迟快速路径和备用回退机制。初步验证表明，本文提出的架构抽象有助于减少多架构带来的维护负担，vDSO在常见的时钟查询场景中带来了可观的延迟改善，而域隔离与驱动加载设计在多核环境下保持了基本的可用性与可恢复性。尽管当前实现仍以最小可运行目标为导向，存在设备支持不全、某些子系统需进一步扩展及更严格形式化验证不足等局限，但本研究为在多架构环境下实现轻量级隔离内核提供了切实的工程路径与实践经验，并为后续在协作调度、用户态中断、vDSO深度优化与更广泛设备支持等方向的研究奠定了基础。

后续工作将围绕三个相互关联的方向展开：

1. 将vsched2的协作调度机制融入域生命周期，并建立可回退的混合调度策略
2. 引入用户态中断以在保持隔离性的前提下缩短I/O与事件处理延迟
3. 在x86_64物理硬件上完成部署与长期验证。

这些扩展将与vDSO的深度优化协同推进，逐步构筑一个在真实多核硬件上兼具严格隔离、低延迟通信和工程可维护性的多架构轻量级隔离内核体系。

参考文献

- [1] Linux kernel community. The kernel's command-line parameters (search for 'kaslr')[EB/OL]. [2026-05-19]. <https://www.kernel.org/doc/html/latest/admin-guide/kernel-parameters.html>.
- [2] Linux kernel community. Linux kernel seccomp filter documentation[EB/OL]. [2026-05-19]. https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html.
- [3] Godones. Alien: A simple operating system implemented in Rust[CP/OL]. [2026-05-19]. <https://github.com/Godones/Alien>.
- [4] Rust OSDev Community. acpi: A library for parsing ACPI tables and AML[CP/OL]. [2026-05-18]. <https://github.com/rust-osdev/acpi>.
- [5] x2apic Developers. x2apic: A Rust interface to the x2apic interrupt architecture (version 0.4.1)[CP/OL]. [2026-05-18]. <https://docs.rs/x2apic/0.4.1/>.
- [6] x86_64 Contributors. x86_64: Support for x86_64 specific instructions, registers, and structures[CP/OL]. [2026-05-18]. https://docs.rs/x86_64/.
- [7] ArceOS Community. ArceOS: A modular operating system framework in Rust[CP/OL]. [2026-05-18]. <https://github.com/arceos/arceos>.
- [8] rosy233333. vdso_crate_template[CP/OL]. [2026-05-18]. https://github.com/rosy233333/vdso_crate_template.
- [9] Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer ‘s Manual Combined Volumes 3A, 3B, 3C, and 3D: System Programming Guide[R/OL]. (2026-03-31)[2026-05-19]. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [10] UEFI Forum. Advanced Configuration and Power Interface (ACPI) Specification, Version 6.5[S/OL]. (2022-08-29)[2026-05-19]. <https://uefi.org/specifications>.
- [11] PCI-SIG. PCI Firmware Specification Revision 3.2[S/OL]. (2015-01-26)[2026-05-19]. <https://pcisig.com/specifications>.
- [12] GNU GRUB Project. Multiboot Specification (version 0.6.96)[S/OL]. [2026-05-19]. <https://www.gnu.org/software/grub/manual/multiboot/multiboot.html>.
- [13] RISC-V International. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20211203[S/OL]. (2021-12-03)[2026-05-19]. <https://riscv.org/technical/specifications/>.
- [14] Linux Kernel Community. vdso(7) — Linux manual page[EB/OL]. (2026-03-20)[2026-05-19]. <https://manpages.debian.org/testing/manpages/vdso.7.en.html>.
- [15] Klein G, Elphinstone K, Heiser G, et al. seL4: formal verification of an OS kernel[C]//Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP 2009). Big Sky, Montana, USA: ACM, 2009: 207-220.
- [16] Liedtke J. On μ -kernel construction[C]//Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP 1995). Copper Mountain Resort, CO: ACM, 1995: 237-250.
- [17] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: securing the foundations of the Rust programming language[J]. Proceedings of the ACM on Programming Languages (POPL), 2018, 2: 1-34.
- [18] Klabnik S, Nichols C. The Rust Programming Language[M]. 2nd ed. San Francisco: No Starch Press, 2023.
- [19] The Rust Project. The Rust Reference[EB/OL]. [2026-05-19]. <https://doc.rust-lang.org/reference/>.

- [20] Kleen A. x86_64: Add vDSO for x86-64 with gettimeofday/clock_gettime/getcpu[EB/OL]. (2007-05-01)[2026-05-21]. <https://lkml.indiana.edu/hypermail/linux/kernel/0705.0/0071.html>.
- [21] Frascino V. Unify vDSOs across more architectures[EB/OL]. (2019-06-21)[2026-05-21]. <https://lwn.net/Articles/791685/>.
- [22] Schnelle S. s390: convert to GENERIC_VDSO[EB/OL]. (2020-08-03)[2026-05-21]. <https://lkml.org/lkml/2020/8/3/622>.
- [23] Larabel M. MSEAL Protection Of System Mappings Merged For Linux 6.15[EB/OL]. (2025-04-04)[2026-05-21]. <https://www.phoronix.com/news/Linux-6.15-MSEAL-System-Mapping>.
- [24] scumjr. dirtycow-vdso: PoC for Dirty COW (CVE-2016-5195) vDSO container escape[EB/OL]. (2016)[2026-05-21]. <https://github.com/scumjr/dirtycow-vdso>.

致 谢

毕业设计进入尾声，回望整个过程，离不开老师的指导和学长的帮助，在此致以由衷的感谢。

感谢我的指导教师陆慧梅老师。从题目确定，展开工作再到论文撰写，陆老师在关键节点上给予了明确而中肯的建议。正是这些及时的纠正，让我在遇到瓶颈时仍能持续推进工作。

感谢实验室的陈林峰学长和罗熙学长。在工作复现、系统实现和反复调试的日子里，学长耐心解答我的困惑，分享自己的开发经验。与学长的讨论和协作不仅帮我扫清了许多技术障碍，也让我对操作系统领域有了更切实的体会。

最后，感谢北京理工大学所提供的学习条件和资源，为本次毕业设计的完成提供了切实的保障。